

Assignment Requirements Summary

<https://chatgpt.com/share/6a70adae-f600-83ea-844d-e169b6506796>

Date: 2026-08-03 · 114 messages

User

Read this assignment brief and summarize the functional requirements, technical requirements, and marking rubric. Identify any requirements that students commonly miss. Do not generate any code yet.

ChatGPT

Here's a structured summary of the assignment based only on the uploaded brief.

Functional Requirements

The application must be a **local-first Todo application** built with **Next.js and SQLite**. It runs locally via Node.js/npm and is intended for a **single user**—there are **no user accounts**.

The required functionality includes:

- **Create tasks**
 - Each task must contain:
 - Title
 - Description
 - Due Date
 - Topic
- **Edit tasks**
 - Changes must persist after refreshing or restarting the application.
- **Archive tasks**
 - Tasks **cannot be deleted**.
 - Archived tasks must no longer appear in the active list but must still be viewable.
- **Task statuses**
 - Exactly three statuses:
 - Todo
 - In-Progress
 - Complete
 - These statuses are fixed and **not user configurable**.
- **Sorting**
 - Users must be able to sort the task list by:
 - Topic
 - Status
 - Due Date
- **Overdue indication**
 - Overdue tasks must be visually indicated.
 - "Overdue" **must not** be implemented as another status.
- **Persistence**

- All data must still exist after the application is closed and restarted.

Technical Requirements

Technology

- Next.js
- SQLite
- Local application (not deployed)

Testing

The repository must contain:

- At least **three tests**
- Tests must exercise **real behaviour**
- All tests must run from **one documented command**

Documentation (Markdown)

The repository must include documentation covering:

1. **Third-Party Code**
 - Every library/package used
 - One-line explanation for why each was chosen
2. **Database Design**
 - Tables
 - Relationships between tables
3. **Running It**
 - Node version
 - Exact install command(s)
 - Exact run command(s)
 - Exact test command(s)
 - Enough information that someone can start from a clean clone without extra instructions

Submission

Submit:

- GitHub repository link
- Documentation files
- AI usage transcripts covering:
 - planning
 - code generation
 - debugging

Marking Rubric

Functional Walkthrough (28 marks)

Worth **4 marks each**, pass/fail.

1. Application installs and runs using only the README.
2. Create a task with all four required fields.
3. Edit a task and verify changes survive reload.
4. Archive a task while keeping it viewable.
5. Sort by topic, status and due date.
6. Overdue task is visibly flagged without becoming a status.
7. Restart application and confirm persistence.

A failed installation walkthrough gets only one additional 10-minute attempt; if that also fails, the walkthrough earns **0/28**.

Rubric (72 marks)

| Category | Weight | Full marks require |

|-----|-----:|-----|

| Documentation | 18 | Complete dependency explanations, accurate database documentation, verified setup instructions including Node version |

| Commit History | 18 | At least 6 meaningful commits across multiple sessions with descriptive commit messages |

| Database Design | 16 | Proper SQLite schema, sensible constraints, archive stored as flag/timestamp, overdue derived rather than stored |

| Testing | 12 | At least 3 deterministic tests using a temporary database, runnable with one command, including archiving or overdue behaviour |

| AI Usage | 8 | Prompt transcripts showing constraints, critical evaluation, corrections, and decisions reflected in the final code

Requirements Students Commonly Miss

The brief contains several details that are easy to overlook:

1. Tasks cannot be deleted

Only **archived**. Archived tasks must remain viewable. Simply deleting rows will lose marks.

2. Overdue is not a status

Many implementations add an "Overdue" status. The brief explicitly forbids this. It must be a visual indication derived from the due date.

3. Overdue should be derived, not stored

The rubric specifically expects overdue to be calculated at read time rather than saved in the database.

4. Archive should be represented in the schema

The rubric expects archive to be stored as something like:

- an archived flag, or

- an archived_at timestamp

Copying rows to another table or deleting them is considered poor design.

5. Tests must use a throwaway database

Tests should not depend on the developer's existing SQLite file. They should be deterministic and isolated.

6. README quality directly affects marks

The application must install from a clean clone using only the documented instructions, including the Node version and all required commands.

7. Commit history is assessed

The rubric expects:

- at least six commits,
- meaningful commit messages,
- work spread across multiple sessions rather than one bulk upload.

8. AI transcripts are graded

Simply including AI-generated code isn't enough. The transcripts should demonstrate that you:

- set constraints,
- reviewed outputs critically,
- rejected or corrected unsuitable suggestions,
- made decisions reflected in the final implementation.

Overall, the assignment emphasizes not just functionality but also software engineering practices: reproducible setup, sound database design, meaningful testing, good version control habits, and documented, thoughtful use of AI.

User

I want to build this in Next.js with SQLite. Help me design the application before writing code. Recommend the folder structure, libraries, database approach, testing framework, and explain why each choice satisfies the assignment requirements. Keep it simple and local-first.

ChatGPT

For this assignment, I'd keep the architecture deliberately small. The rubric rewards correctness, persistence, testing, documentation, and database design—not enterprise patterns. A simple, conventional Next.js project will be easier to complete and easier for the marker to understand.

Overall Architecture



SQLite Database



Task Repository

Everything runs locally.

No authentication.

No external services.

No cloud database.

This matches the brief's "local-first" requirement.

Recommended Folder Structure

```
todo-app/
├── app/
│   ├── page.tsx                # Task list
│   ├── create/
│   │   └── page.tsx
│   ├── edit/
│   │   └── [id]/
│   │       └── page.tsx
│   ├── archived/
│   │   └── page.tsx
│   ├── actions.ts              # Server Actions
│   └── globals.css
├── components/
│   ├── TaskCard.tsx
│   ├── TaskForm.tsx
│   ├── SortBar.tsx
│   └── StatusBadge.tsx
├── lib/
│   ├── db.ts                   # SQLite connection
│   ├── taskRepository.ts       # Database queries
│   └── overdue.ts              # Helper functions
├── database/
│   ├── schema.sql
│   └── todo.db
├── tests/
│   ├── create.test.ts
│   ├── archive.test.ts
│   └── overdue.test.ts
├── docs/
│   ├── database.md
│   └── third-party.md
```

```
|   └─ running.md
|
|   └─ README.md
|   └─ package.json
|   └─ tsconfig.json
```

This keeps database logic separate from UI while remaining very easy to understand.

Recommended Libraries

1. Next.js

Purpose

Application framework.

Why

- Required by the brief
- Routing included
- React included
- Easy local development

2. SQLite

Purpose

Persistent local database.

Why

The assignment explicitly requires SQLite.

It stores data locally and survives application restarts.

3. better-sqlite3

Purpose

SQLite driver.

Why I'd choose it

- Very simple API
- Fast
- Synchronous (perfect for a local desktop-style app)
- No database server required

This is probably the simplest SQLite library for this assignment.

4. TypeScript

Purpose

Static typing.

Why

Comes with Next.js and reduces mistakes.

5. Vitest

Purpose

Testing framework.

Why

The assignment requires three real tests.

Vitest is:

- lightweight
- fast
- simple
- works well with TypeScript

6. Testing Library

Purpose

Testing UI behaviour when needed.

Why

Allows tests such as:

- creating a task
- editing
- archiving

without relying on implementation details.

7. ESLint

Included with Next.js.

Helps catch mistakes.

Database Design

The assignment only requires tasks.

One table is enough.

```
Task
-----
id
title
```

```
description
topic
due_date
status
archived
created_at
updated_at
```

Status

Store as one of

```
Todo
In-Progress
Complete
```

Exactly as required.

Archived

Store as

```
BOOLEAN archived
```

or

```
archived_at TIMESTAMP
```

The rubric specifically prefers a flag or timestamp.

Do **not** move archived tasks to another table.

Overdue

Never store:

```
overdue = true
```

Instead compute

```
due_date < today
AND
status != Complete
```

every time tasks are displayed.

This exactly matches the rubric.

Repository Pattern

Instead of writing SQL throughout the application:

```
UI
```

```
↓
```


Task Repository

↓

SQLite

Example responsibilities:

createTask()

updateTask()

archiveTask()

getActiveTasks()

getArchivedTasks()

getSortedTasks()

Advantages

- keeps SQL in one place
- easier testing
- easier maintenance

UI Pages

Home

Shows

- active tasks
- sorting controls

Create

Simple task form.

Edit

Loads existing task.

Updates database.

Archived

Displays archived tasks only.

Sorting

Support exactly

- Topic
- Status
- Due Date

No need for filtering unless you want it.

Task Status

Keep these fixed.

Todo

In-Progress

Complete

Dropdown only.

Do not let users create statuses.

Testing Strategy

The rubric specifically mentions:

- three tests
- real behaviour
- throwaway database

I'd write tests like:

Test 1

Create task

Verify

- exists
- all four fields saved

Test 2

Archive task

Verify

- removed from active list
- appears in archived list

Test 3

Overdue rule

Insert task

with yesterday's due date

Verify

```
isOverdue(task)
```

returns true.

No database column involved.

Persistence

Store

```
database/todo.db
```

Every restart:

```
Open SQLite
```

```
↓
```

```
Load tasks
```

```
↓
```

```
Done
```

Nothing special needed.

Documentation Files

The brief requires three sections, so I would create:

```
docs/
```

```
database.md
```

```
third-party.md
```

```
running.md
```

Each can later be linked from the README.

README

Include

- project overview
- Node version
- install command

- run command
- test command

The walkthrough begins with the README, so it should be complete enough that someone can start the application from a clean clone without guessing any steps.

Why this design fits the assignment

Requirement	Design choice
Next.js	Uses the App Router with a simple page structure
SQLite	Local <code>todo.db</code> database for persistence
Local-first	No cloud services or authentication
Create/Edit/Archive	Single Task table with an archived flag
Persistence	SQLite database survives restarts
Fixed statuses	Store only <code>Todo</code> , <code>In-Progress</code> , and <code>Complete</code>
Overdue	Derived from due date at runtime, not stored
Sorting	Database queries or simple ordering by topic, status, or due date
Three tests	Vitest with a temporary SQLite database exercising real behavior
Documentation	Separate Markdown files for dependencies, database design, and running instructions

This design stays intentionally small, aligns closely with the assignment brief, and avoids unnecessary complexity while meeting the functional and rubric expectations.

User

Design a SQLite database schema for this application. Explain the tables, columns and constraints

ChatGPT

Based on the assignment brief, a **single `Task` table** is sufficient. The rubric actually favors a simple schema where archived tasks remain in the same table and overdue status is **derived** rather than stored.

Database Design

The database contains one table:

Task

Each row represents a single task created by the user.

There is no need for separate tables for users or statuses because:

- the application is single-user,
- statuses are fixed,
- topics are simply text entered for each task.

Task Table

Column	Type	Constraints	Purpose
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique identifier for each task
title	TEXT	NOT NULL	Short task title
description	TEXT	NOT NULL	Detailed task description
topic	TEXT	NOT NULL	Category or subject of the task
due_date	DATE (stored as TEXT in ISO format)	NOT NULL	Deadline for the task
status	TEXT	NOT NULL, CHECK constraint	Current task status
archived	INTEGER	NOT NULL DEFAULT 0	Indicates whether the task has been archived
created_at	DATETIME	NOT NULL	When the task was created
updated_at	DATETIME	NOT NULL	Last modification time

Explanation of Each Column

id

INTEGER PRIMARY KEY AUTOINCREMENT

Purpose:

- uniquely identifies each task
- used when editing or archiving a task

Why:

Every table should have a stable primary key.

title

TEXT NOT NULL

Stores the task title.

Example:

Finish Lab Report

The assignment specifies that every task must have a title, so it should not allow NULL values.

description

TEXT NOT NULL

Stores additional information.

Example:

Complete sections 3–5 before submission.

Required by the assignment.

topic

TEXT NOT NULL

Represents the category of the task.

Examples:

University
Personal
Work
Shopping

The assignment allows sorting by topic, so storing it directly as text is sufficient.

due_date

DATE

SQLite stores dates as text, typically in ISO format (YYYY-MM-DD).

Example:

2026-08-15

This field is used for:

- displaying deadlines
- sorting by due date
- determining whether a task is overdue

status

TEXT

Allowed values:

Todo
In-Progress
Complete

A CHECK constraint should restrict the column to these three values because the assignment specifies that statuses are fixed and not user-customisable.

archived

INTEGER DEFAULT 0

SQLite has no native Boolean type, so:

0 = active
1 = archived

Why this design?

The assignment states tasks cannot be deleted, only archived, and the rubric explicitly prefers archive to be represented by a flag or timestamp rather than copying rows elsewhere.

created_at

Stores when the task was first created.

Useful for:

- auditing
- debugging
- future extensions

Although not required by the brief, it is a sensible design choice.

updated_at

Stores when the task was last modified.

Updated whenever:

- title changes
- description changes
- topic changes
- due date changes
- status changes
- task is archived

Again, not required but good database practice.

Constraints

Primary Key

```
PRIMARY KEY(id)
```

Ensures every task has a unique identifier.

NOT NULL

Use NOT NULL for:

- title
- description
- topic
- due_date
- status
- archived
- created_at

- updated_at

This prevents incomplete records from being stored.

Status Constraint

Restrict status values to exactly:

```
Todo
In-Progress
Complete
```

This ensures users cannot create arbitrary statuses and enforces the assignment requirement.

Default Value

For archived:

```
DEFAULT 0
```

Every newly created task starts as active.

What Should NOT Be Stored

Overdue

The rubric specifically expects overdue to be **derived**, not stored in the database.

Instead of a column like:

```
overdue = 1
```

the application should determine if a task is overdue by checking whether:

- the current date is later than due_date, and
- the task is not marked as Complete.

This keeps the database consistent, since a task can become overdue simply as time passes.

Relationships

There are **no relationships** in this design because the application only requires a single Task entity. This still satisfies the assignment's requirement to document the database design and relationships—your documentation can state that the schema consists of one table and therefore has no inter-table relationships.

Entity Relationship Diagram (ERD)

```
+-----+
```


Task	
PK id : INTEGER	
title : TEXT	
description : TEXT	
topic : TEXT	
due_date : DATE	
status : TEXT	
archived : BOOLEAN (0/1)	
created_at : DATETIME	
updated_at : DATETIME	

This schema is intentionally simple, fully supports the required features (create, edit, archive, sorting, persistence), and aligns with the rubric's expectations by keeping archived tasks in the same table and deriving overdue status rather than storing it.

User

Recommend only the npm packages actually needed for this project. For each package explain what it does and why it was chosen. Avoid unnecessary dependencies.

ChatGPT

For this assignment, you only need a handful of dependencies. The goal is to keep the project simple, easy to understand, and aligned with the rubric. Everything below has a clear purpose; anything beyond this is optional rather than necessary.

Package	Type	What it does	Why it was chosen
next	Production	The Next.js framework for building the application.	Required by the assignment. Provides routing, server-side functionality, and the React framework in one package.
react	Production	Builds the user interface using components.	Required dependency of Next.js.
react-dom	Production	Renders React components in the browser.	Required dependency of Next.js.
better-sqlite3	Production	Provides direct access to the SQLite database.	A lightweight, synchronous SQLite library that is simple to use in a local-first application. It requires no separate database server and works well with Next.js server code.
typescript	Development	Adds static typing to the project.	Included by default with modern Next.js projects. Improves reliability and makes the code easier to maintain.
vitest	Development	Runs automated tests.	A fast, lightweight testing framework that easily satisfies the assignment's requirement for at least three runnable tests.
@testing-library/react	Development	Tests React components by interacting with them as a user would.	Encourages testing application behaviour rather than implementation details, matching the rubric's emphasis on "real behaviour".
@testing-library/jest-dom	Development	Adds useful assertions for UI tests.	Makes component tests clearer and more readable (for example, checking whether elements are visible or contain expected text).
eslint	Development	Analyses the code for common errors and style issues.	Included

with Next.js and helps maintain code quality throughout development. |

Packages you do not need

To keep the project focused, I would avoid adding these unless a genuine need arises:

- **Prisma** – adds an ORM layer and migration system that is unnecessary for a single-table SQLite database.
- **Drizzle ORM** – another good ORM, but unnecessary complexity for this assignment.
- **Redux, Zustand, MobX** – state management libraries are excessive for a small local Todo app.
- **Tailwind CSS** – useful for styling, but not required. Plain CSS or CSS Modules are sufficient unless you specifically want Tailwind.
- **Formik / React Hook Form** – the task form is simple enough to manage with React's built-in state.
- **date-fns / dayjs / moment** – overdue calculations only require comparing dates, which can be handled with JavaScript's built-in Date object.
- **Axios** – there are no external APIs to call; Next.js server actions or route handlers communicate directly with SQLite.
- **UUID libraries** – SQLite's auto-incrementing primary key is sufficient.
- **Authentication libraries** – the brief specifies a single-user local application with no user accounts.

Minimal dependency list

If you aim for the smallest practical project, the dependency list is simply:

Production

- next
- react
- react-dom
- better-sqlite3

Development

- typescript
- vitest
- @testing-library/react
- @testing-library/jest-dom
- eslint

This set is sufficient to build the application, persist data in SQLite, implement the required functionality, and meet the assignment's testing requirements without introducing unnecessary dependencies.

User

Show me how to create a new Next.js project using the App Router and TypeScript. Then explain where each major file should go before writing any code.

ChatGPT

1. Create a New Next.js Project

Open a terminal and run:

```
npx create-next-app@latest todo-app
```

When prompted, choose:

```
✓ Would you like to use TypeScript? → Yes
✓ Would you like to use ESLint? → Yes
✓ Would you like to use Tailwind CSS? → No
✓ Would you like your code inside a src/ directory? → No
✓ Would you like to use App Router? → Yes
✓ Would you like to use Turbopack for next dev? → Yes
✓ Would you like to customize the import alias? → No
```

Then move into the project:

```
cd todo-app
```

Install the additional packages you'll need:

```
npm install better-sqlite3
```

Install the testing packages:

```
npm install -D vitest @testing-library/react @testing-library/jest-dom
```

Start the development server:

```
npm run dev
```

Visit:

```
http://localhost:3000
```

If you see the default Next.js page, the project is ready.

2. Recommended Project Structure

Before writing any code, create the folders so the project stays organized.

```
todo-app/
|
├── app/
|
├── components/
|
├── lib/
|
├── database/
|
├── tests/
|
├── docs/
|
├── public/
|
├── package.json
├── tsconfig.json
└── README.md
```

3. Purpose of Each Folder

app/

This is the App Router.

Every page of your application lives here.

Example:

```
app/  
|  
├─ page.tsx  
├─ create/  
├─ edit/  
├─ archived/  
└─ actions.ts
```

page.tsx

The application's home page.

It will display

- task list
- sort controls
- navigation

create/

Contains the page for creating a task.

```
app/create/page.tsx
```

edit/

Contains the page for editing a task.

```
app/edit/[id]/page.tsx
```

[id] means the page receives the task ID from the URL.

archived/

Contains the page showing archived tasks.

actions.ts

A good place for Next.js Server Actions.

Responsibilities later include

- create task
- update task

- archive task

Keeping them together avoids mixing database logic into UI components.

components/

Reusable UI components.

Example:

```
components/  
  
TaskForm.tsx  
  
TaskCard.tsx  
  
SortBar.tsx  
  
StatusBadge.tsx
```

Each component has one job.

For example

TaskCard

↓

Displays one task.

TaskForm

↓

Handles task input.

SortBar

↓

Lets the user choose how to sort.

lib/

Contains application logic.

Example

```
lib/  
  
db.ts  
  
taskRepository.ts  
  
overdue.ts
```

db.ts

Responsible for opening the SQLite database.

Only this file should know how the database connection works.

taskRepository.ts

Contains database operations such as

- creating tasks
- editing tasks
- loading tasks
- archiving tasks

The UI should never execute SQL directly.

overdue.ts

Contains helper functions.

Example responsibility

Determine whether a task is overdue.

Keeping this logic separate makes it easier to test.

database/

Contains database files.

```
database/  
  
schema.sql  
  
todo.db
```

schema.sql

Defines the database tables.

Useful for documentation and recreating the database.

todo.db

The SQLite database.

Created automatically when the application runs.

This provides persistence after restarting the application, as required by the brief.

tests/

Contains automated tests.

Example

```
tests/  
  
create.test.ts  
  
archive.test.ts  
  
overdue.test.ts
```

These correspond well to the assignment's requirement for at least three tests that exercise real behaviour.

docs/

Contains the required documentation.

Suggested files:

```
docs/  
  
database.md  
  
third-party.md  
  
running.md
```

These map directly to the documentation sections required in the assignment:

- Third-Party Code
- Database Design
- Running It

public/

Stores static assets.

Examples:

```
public/  
  
logo.png  
  
icons/  
  
images/
```

Files placed here are served directly by Next.js.

README.md

The first file the marker is likely to read.

It should eventually contain:

- project overview
- Node version
- installation commands
- run command
- test command

The functional walkthrough begins by following the README from a clean clone.

How Everything Fits Together



This flow keeps responsibilities separate:

- **`app/`** handles pages and routing.
- **`components/`** contains reusable UI pieces.
- **`lib/`** contains business logic and database access.
- **`database/`** stores the schema and SQLite database.
- **`tests/`** contains automated tests.
- **`docs/`** contains the required assignment documentation.

This structure is intentionally small, easy to navigate, and aligns well with both the assignment's functional requirements and its marking rubric.

User

i ran npx create, asking react compiler?

ChatGPT

If create-next-app is asking:

Would you like to use React Compiler? > No / Yes

Choose:

→ No

Why?

For this assignment, React Compiler provides no real benefit. It's an experimental optimization feature and adds another variable when you want a straightforward, stable project.

Your choices should be:

```
✓ TypeScript → Yes
✓ ESLint → Yes
✓ React Compiler → No
✓ Tailwind CSS → No
✓ src/ directory → No
✓ App Router → Yes
✓ Turbopack → Yes
✓ Import alias → No
```

This gives you a clean, minimal Next.js project that's ideal for the SQLite todo app.

User

npm error code EALLOWSCRIPTS

npm error --allow-scripts is not allowed in project-scoped installs. Add the entries to the "allowScripts" field in package.json, or to .npmrc, instead.

npm error A complete log of this run can be found in: /home/zidan/.npm/logs/2026-08-03T155024692Z-debug-0.log

Aborting installation.

npm install has failed. got this error

ChatGPT

This isn't a Next.js problem—it's an **npm security policy** on your machine.

The error:

```
EALLOWSCRIPTS
--allow-scripts is not allowed in project-scoped installs.
```

usually means you have an npm configuration (often from tools like @lavamoat/allow-scripts or a strict corporate/security setup) that blocks packages from running install scripts. Unfortunately, **better-sqlite3** requires an install script to build or download its native binary.

Step 1: Check your npm configuration

Run:

```
npm config list
```

and

```
npm config get allow-scripts
```

Also check if you have an `.npmrc` file:

```
cat ~/.npmrc
```

and, if you're inside the project:

```
cat .npmrc
```

Step 2: Check your package.json

If the project was created, open `package.json` and see if it contains something like:

```
"allowScripts": {  
  ...  
}
```

or if there are dependencies related to `@lavamoat/allow-scripts`.

Step 3: Tell me exactly when it happened

Did this error occur:

- **A.** During `npx create-next-app@latest todo-app`
- **B.** During `npm install better-sqlite3`
- **C.** During another `npm install` command?

The fix depends on which command triggered it.

Step 4: Share a little more information

Please paste the output of:

```
node -v  
npm -v  
  
npm config list  
  
cat ~/.npmrc
```

If there's no `.npmrc`, it will say "No such file or directory"—that's fine.

With that information, I can pinpoint exactly what's enforcing the `allow-scripts` policy and tell you the safest way to remove or configure it.

User

during `npx create node -v`

`npm -v`

`npm config list`

`cat ~/.npmrc`

`v24.18.1`

`11.16.0`

`; "user" config from /home/zidan/.npmrc`

`allow-scripts =`

`["core-js,esbuild,keytar,koffi,onnxruntime-node,protobufjs,sharp,tls-client-node,@parcel/watcher,@swc`

`; node bin location = /home/zidan/.nvm/versions/node/v24.18.1/bin/node`

```
; node version = v24.18.1
; npm local prefix = /home/zidan/Desktop/SDP/todo-app
; npm version = 11.16.0
; cwd = /home/zidan/Desktop/SDP/todo-app
; HOME = /home/zidan
; Run npm config ls -l to show all defaults.
allow-scripts=core-js,esbuild,keytar,koffi,onnxruntime-node,protobufjs,sharp,tls-client-node,@parcel/w
```

ChatGPT

The important clue is this:

```
cwd = /home/zidan/Desktop/SDP/todo-app
```

That means you **already created and entered the project directory** before running create-next-app, or the first attempt partially created it.

The real issue is your npm configuration:

```
allow-scripts=core-js,esbuild,...,better-sqlite3,...
```

With **npm 11**, the old allow-scripts flag is no longer accepted during project-scoped installs. Something is invoking:

```
npm install --allow-scripts
```

which npm 11 rejects.

First, let's identify what's doing it.

Run these commands inside the project:

```
cat package.json
```

and

```
ls -la
```

Also check whether a project .npmrc exists:

```
cat .npmrc
```

If there is no .npmrc, that's fine.

I also want you to check which create-next-app you're running:

```
npx create-next-app@latest --version
```

and

```
which npx
```

My suspicion

From your previous conversations, I know you've installed developer tooling like **OmniRoute** globally. One of your global npm packages or npm configuration is likely wrapping npm install and injecting the obsolete --allow-scripts option.

It is **not** better-sqlite3 causing this—the error occurs before your project dependencies are even installed.

Once you paste:

- `package.json`
- `ls -la`
- `cat .npmrc` (inside the project)
- `npx create-next-app@latest --version`

I can tell you exactly what needs to be changed.

User

```
name "todo-app"
version "0.1.0"
private true
scripts
dev "next dev"
build "next build"
start "next start"
lint "eslint"
dependencies
next "16.2.12"
react "19.2.4"
react-dom "19.2.4"
devDependencies
@types/node "^20"
@types/react "^19"
@types/react-dom "^19"
eslint "^9"
eslint-config-next "16.2.12"
typescript "^5"
ls -la
total 16
drwxrwxr-x 4 zidan zidan 4096 Aug  3 17:50 .
drwxrwxr-x 3 zidan zidan 4096 Aug  3 17:08 ..
drwxrwxr-x 7 zidan zidan 4096 Aug  3 17:08 .git
drwxrwxr-x 4 zidan zidan 4096 Aug  3 17:50 todo-app
zidan@zidan-Latitude-5490:~/Desktop/SDP/todo-app$ cat .npmrc
cat: .npmrc: No such file or directory
zidan@zidan-Latitude-5490:~/Desktop/SDP/todo-app$ npx create-next-app@latest
--version
16.2.12. I did clone an empty github repo before running create
```

ChatGPT

Thanks, this narrows it down.

The project itself looks completely normal. The problem is **not** your `package.json` or the GitHub repository.

What stands out is:

- You're using **npm 11.16.0**
- You have a global **allow-scripts** configuration in ~/.npmrc
- The error occurred during create-next-app, when it runs npm install

This is a known compatibility issue: some versions of create-next-app still invoke npm install in a way that conflicts with npm 11's new handling of allow-scripts.

First, check if the project actually installed

Go into the generated project:

```
cd ~/Desktop/SDP/todo-app/todo-app
```

Then run:

```
ls
```

Do you see a node_modules folder?

Also run:

```
npm run dev
```

If that works, you can ignore the aborted create-next-app message—the project was created successfully.

If node_modules is missing

Try installing manually:

```
cd ~/Desktop/SDP/todo-app/todo-app  
npm install
```

If **that** also gives EALLOWSCRIPTS, then the issue is definitely your npm configuration rather than Next.js.

I suspect a global npm hook

Please run:

```
npm config ls -l | grep allow
```

and

```
npm config get ignore-scripts
```

Also check whether you have the @lavamoat/allow-scripts package installed globally:

```
npm ls -g --depth=0
```

I'm looking for something like:

- @lavamoat/allow-scripts
- allow-scripts
- any other package related to install script policies.

Your directory layout

One small thing I noticed:

```
~/Desktop/SDP/todo-app
└─ todo-app
```

So you accidentally created a project **inside** the empty Git repository.

I'd actually recommend deleting the inner folder and recreating it properly, or creating the project directly in the existing repository using:

```
npx create-next-app@latest .
```

(with the `.`), **after** we fix the npm issue.

Let's first determine whether the problem is coming from a global package like LavaMoat or another npm wrapper. The output of `npm ls -g --depth=0` will usually reveal it.

User

```
npm config ls -l | grep allow
```

```
allow-directory = "all"
```

```
allow-file = "all"
```

```
allow-git = "all"
```

```
allow-remote = "all"
```

```
allow-same-version = false
```

```
; allow-scripts = [""] ; overridden by user
```

```
allow-scripts-pending = false
```

```
allow-scripts-pin = true
```

```
dangerously-allow-all-scripts = false
```

```
strict-allow-scripts = false
```

```
allow-scripts =
```

```
["core-js,esbuild,keytar,koffi,onnxruntime-node,protobufjs,sharp,tls-client-node,@parcel/watcher,@swc
```

```
zidan@zidan-Latitude-5490:~/Desktop/SDP/todo-app/todo-app$ npm config get
```

```
ignore-scripts
```

```
false
```

```
zidan@zidan-Latitude-5490:~/Desktop/SDP/todo-app/todo-app$ npm ls -g --depth=0
```

```
/home/zidan/.npm/versions/node/v24.18.1/lib
```

```
└─ corepack@0.35.0
```

```
└─ npm@11.16.0
```

```
└─ omniroute@3.8.49
```

ChatGPT

That explains it.

The culprit is almost certainly **OmniRoute**.

You have:

```
omniroute@3.8.49
```

installed globally, and your `~/.npmrc` contains:

```
allow-scripts = ["core-js,...,better-sqlite3,omniroute"]
```

This isn't something Next.js creates—it's almost certainly added by OmniRoute (or its installer) to enforce an allow-list for npm install scripts.

The error:

```
EALLOWSCRIPTS  
--allow-scripts is not allowed in project-scoped installs.
```

suggests something is still invoking npm with the deprecated `--allow-scripts` CLI flag, which npm 11 rejects.

First, confirm it's OmniRoute

Run:

```
omniroute doctor
```

and

```
omniroute --version
```

Then check if OmniRoute added shell aliases:

```
type npm  
type npx
```

The expected output should be:

```
npm is /home/zidan/.nvm/versions/node/v24.18.1/bin/npm  
npx is /home/zidan/.nvm/versions/node/v24.18.1/bin/npx
```

If either is an alias or shell function, paste the output.

Quick workaround

Since you're not using OmniRoute for this assignment, the easiest fix is to temporarily remove it:

```
npm uninstall -g omniroute
```

Then remove the allow-scripts configuration:

```
npm config delete allow-scripts
```

Verify it's gone:

```
npm config get allow-scripts
```

It should return undefined or an empty value.

Then recreate the project (or reinstall dependencies).

If you need OmniRoute later

You can reinstall it after finishing the assignment:

```
npm install -g omniroute
```

One more thing

I also recommend fixing your directory layout.

Right now you have:

```
~/Desktop/SDP/todo-app    ← Git repository
└─ todo-app              ← Next.js project
```

If the Git repository was intended to hold the project, it's cleaner to create the Next.js app directly in the repository root:

```
cd ~/Desktop/SDP/todo-app
rm -rf todo-app # only if you're happy to recreate it

npx create-next-app@latest .
```

Using the `.` creates the project in the current directory instead of creating a nested `todo-app` folder.

Before uninstalling OmniRoute, I'd like to see the output of:

```
type npm
type npx
```

That will confirm whether OmniRoute has modified how `npm` or `npx` are being invoked.

User

```
type npm
```

```
type npx
```

```
npm is hashed (/home/zidan/.nvm/versions/node/v24.18.1/bin/npm)
```

```
npx is hashed (/home/zidan/.nvm/versions/node/v24.18.1/bin/npx)
```

ChatGPT

That's actually good news.

Your `npm` and `npx` are the real executables from NVM—they're **not** being aliased or wrapped.

So the issue is most likely a compatibility bug between:

- **Node 24.18.1**
- **npm 11.16.0**
- **create-next-app 16.2.12**
- the `allow-scripts` configuration in npm 11

Before changing anything, let's confirm

Run:

```
npm install
```

inside:

```
~/Desktop/SDP/todo-app/todo-app
```

If ``npm install`` **succeeds**, then the problem is **only** with `create-next-app` and you can continue using the project.

If ``npm install`` **fails with the same error**, then we need to remove the `allow-scripts` configuration.

I would also check this

Run:

```
npm config delete allow-scripts
```

This only removes that one configuration entry from your `~/.npmrc`. It won't uninstall Node, npm, or OmniRoute.

Then verify:

```
npm config get allow-scripts
```

It should return:

```
undefined
```

Now try:

```
npm install
```

again.

If that fixes it

The issue was the npm configuration. If OmniRoute needs it later, you can regenerate it by reinstalling or reconfiguring OmniRoute.

If it still doesn't work

Then this is likely a regression in npm 11. In that case, I'd recommend using the current LTS versions for coursework:

- **Node 22 LTS**
- **npm 10**

You already use **nvm**, so switching is easy:

```
nvm install 22  
nvm use 22
```

Then verify:

```
node -v  
npm -v
```

and recreate the project.

At this point, I think the quickest path is:

1. `npm config delete allow-scripts`
2. `npm install`

Can you tell me what happens when you run ``npm install`` in the `todo-app` directory after deleting `allow-scripts`? That will tell us whether the project is already usable or whether we need to switch Node/npm versions.

User

created the app successfully and fixed folder structure

ChatGPT

Great! That was the hardest part. Now you have a clean starting point.

Next steps

From your project root, install only the packages you'll actually use:

```
npm install better-sqlite3
```

```
npm install -D vitest @testing-library/react @testing-library/jest-dom
```

Then verify the project still runs:

```
npm run dev
```

You should see the default Next.js page at <http://localhost:3000>.

Create the project folders

Before writing any code, I'd create this structure:

```
todo-app/  
├── app/  
├── components/  
├── lib/  
├── database/  
├── tests/  
├── docs/  
└── public/
```

Inside database/, create:

```
database/  
├── schema.sql  
└── todo.db (this will be created automatically later)
```

Inside docs/, create:

```
database.md  
third-party.md  
running.md
```

These map directly to the documentation required by the assignment.

A suggested implementation order

To make development smooth and produce a good commit history for the rubric, I'd build the project in this sequence:

1. ☐ Create the Next.js project (done)
2. Install dependencies
3. Create the folder structure
4. Design and create the SQLite database
5. Connect Next.js to SQLite
6. Implement "Create Task"
7. Display the task list
8. Implement "Edit Task"
9. Implement "Archive Task"
10. Add sorting (topic, status, due date)
11. Add overdue highlighting (derived, not stored)
12. Write the three required tests
13. Complete the documentation
14. Review everything from a clean clone

This order naturally gives you meaningful commits instead of one large commit, which aligns well with the marking rubric.

I recommend we tackle it one feature at a time, starting with **setting up the SQLite database and database connection**, since every other feature depends on it.

User

give the cmds to create the folders and files needed

ChatGPT

From the **root of your Next.js project**, you can create everything with these commands.

Create the folders

```
mkdir -p components lib database tests docs app/create app/edit/[id] app/archived
```

Create the documentation files

```
touch docs/database.md docs/third-party.md docs/running.md
```

Create the database schema file

```
touch database/schema.sql
```

> **Don't create `todo.db` manually.** It should be created automatically by SQLite the first time your application connects to it.

(Optional) Create placeholder component files

```
touch components/TaskCard.tsx \  
  components/TaskForm.tsx \  
  components/SortBar.tsx \  
  components/StatusBadge.tsx
```

(Optional) Create placeholder library files

```
touch lib/db.ts \  
  lib/taskRepository.ts \  
  lib/overdue.ts
```

(Optional) Create placeholder test files

```
touch tests/create.test.ts \  
      tests/archive.test.ts \  
      tests/overdue.test.ts
```

Check the structure

Run:

```
tree -L 2
```

You should have something similar to:

```
todo-app/  
├── app/  
│   ├── archived/  
│   ├── create/  
│   ├── edit/  
│   ├── favicon.ico  
│   ├── globals.css  
│   ├── layout.tsx  
│   └── page.tsx  
├── components/  
│   ├── SortBar.tsx  
│   ├── StatusBadge.tsx  
│   ├── TaskCard.tsx  
│   └── TaskForm.tsx  
├── database/  
│   └── schema.sql  
├── docs/  
│   ├── database.md  
│   ├── running.md  
│   └── third-party.md  
├── lib/  
│   ├── db.ts  
│   ├── overdue.ts  
│   └── taskRepository.ts  
├── public/  
├── tests/  
│   ├── archive.test.ts  
│   ├── create.test.ts  
│   └── overdue.test.ts  
├── package.json  
├── README.md  
└── tsconfig.json
```

This structure is clean, minimal, and provides places for all the functionality and documentation required by the assignment.

User

```
├── app  
│   ├── archived  
│   ├── create  
│   └── edit
```

```
|   ├── favicon.ico
|   ├── globals.css
|   ├── layout.tsx
|   ├── page.module.css
|   └── page.tsx
├── components
|   ├── SortBar.tsx
|   ├── StatusBadge.tsx
|   ├── TaskCard.tsx
|   └── TaskForm.tsx
├── database
|   └── schema.sql
├── docs
|   ├── database.md
|   ├── running.md
|   └── third-party.md
├── eslint.config.mjs
├── lib
|   ├── db.ts
|   ├── overdue.ts
|   └── taskRepository.ts
├── next.config.ts
├── next-env.d.ts
├── node_modules
├── package.json
├── package-lock.json
├── public
|   ├── file.svg
|   ├── globe.svg
|   ├── next.svg
|   ├── vercel.svg
|   └── window.svg
├── README.md
├── tests
|   ├── archive.test.ts
|   ├── create.test.ts
|   └── overdue.test.ts
└── tsconfig.json
```

327 directories, 31 files create a .gitignore

ChatGPT

A good .gitignore for this project should ignore build artifacts, dependencies, environment files, logs, your local SQLite database, and IDE files, while **keeping** database/schema.sql

tracked.

Create a .gitignore in the project root:

```
touch .gitignore
```

Then add:

```
:::writing{variant="document" id="69581"}
```

```
# Dependencies
node_modules/

# Next.js
.next/
out/

# Production
build/

# Logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*

# Environment variables
.env
.env.local
.env.development.local
.env.test.local
.env.production.local

# SQLite database
database/*.db
database/*.sqlite
database/*.sqlite3

# macOS
.DS_Store

# Linux
*~

# Windows
Thumbs.db

# IDEs
.vscode/
.idea/

# Coverage
coverage/

# TypeScript
```

```
*.tsbuildinfo
```

```
:::
```

Why ignore the database?

Your repository should contain the **schema**, not your personal database containing tasks.

Your repository should look like:

```
database/
├── schema.sql      □ tracked
└── todo.db        □ ignored
```

This lets anyone clone the project and generate their own database while still seeing the intended schema.

Finally, check what Git will track:

```
git status
```

You should see files like `schema.sql`, `package.json`, and your source files, but **not** `node_modules`, `.next`, or `database/todo.db`.

User

now lets do step4

ChatGPT

Perfect. **Step 4 is designing and creating the SQLite database.** We'll do this before connecting it to Next.js, so we have a solid foundation.

Step 4.1: Final Database Design

We'll use a **single `Task` table**.

Column	Type	Purpose
id	INTEGER	Unique task ID
title	TEXT	Task title
description	TEXT	Task description
topic	TEXT	Task category
due_date	TEXT	Due date (ISO format: YYYY-MM-DD)
status	TEXT	Todo, In-Progress, or Complete
archived	INTEGER	0 = active, 1 = archived
created_at	TEXT	Creation timestamp
updated_at	TEXT	Last updated timestamp

Constraints

- `id` → Primary Key with Auto Increment
- All fields except `id` are NOT NULL
- `status` is limited to:
 - Todo
 - In-Progress
 - Complete

- archived defaults to 0

This satisfies the brief and the rubric's expectations for database design.

Step 4.2: Populate database/schema.sql

Replace the contents of database/schema.sql with:

```
CREATE TABLE IF NOT EXISTS Task (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  description TEXT NOT NULL,
  topic TEXT NOT NULL,
  due_date TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('Todo', 'In-Progress', 'Complete')),
  archived INTEGER NOT NULL DEFAULT 0 CHECK (archived IN (0, 1)),
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);
```

Why TEXT for dates?

SQLite doesn't have a dedicated DATE type. Storing dates in ISO 8601 format (YYYY-MM-DD for due_date, ISO timestamps for created_at and updated_at) allows them to sort correctly and makes comparisons straightforward.

Step 4.3: What the Database Will Look Like

Example record:

id	title	description	topic	due_date	status	archived
1	Finish Lab	Complete database schema	University	2026-08-10	Todo	0

If archived later:

archived
1

The task stays in the database—it is simply excluded from the active task list.

Step 4.4: What We Will Not Store

We will **not** add an overdue column.

Instead, the application will determine whether a task is overdue by comparing the current date with due_date (and typically only considering tasks that are not Complete). This matches the rubric's requirement that overdue be derived rather than stored.

Step 4.5: Commit

Once you've saved `schema.sql`, this is a good point for your next Git commit:

```
git add .
git commit -m "Design SQLite database schema for tasks"
```

After this, **Step 5** will be to install and configure `better-sqlite3`, create `lib/db.ts`, and have the application automatically create the database from `schema.sql` the first time it runs.

User

what model am i using, need to specify in commits

ChatGPT

You're chatting with **ChatGPT based on GPT-5.5**.

For your AI usage transcripts or commit messages, you can accurately describe it as:

- **OpenAI ChatGPT (GPT-5.5)**

For example, in your AI documentation you could write:

> **AI Model:** OpenAI ChatGPT (GPT-5.5)

or

> **Tool Used:** ChatGPT (GPT-5.5, OpenAI)

For commit messages, you generally don't need to mention the model unless your lecturer specifically wants AI-assisted commits. A good commit message would simply describe the work completed, for example:

```
Design SQLite database schema for task management
```

If you're maintaining an AI transcript for the assignment, it's useful to include entries like:

Date	Model	Purpose
-----	-----	-----
2026-08-03	ChatGPT (GPT-5.5)	Planned project structure and technology stack
2026-08-03	ChatGPT (GPT-5.5)	Designed SQLite schema and database constraints
2026-08-03	ChatGPT (GPT-5.5)	Reviewed assignment requirements and marking rubric

That level of detail should satisfy the assignment's requirement to submit AI planning and design transcripts.

User

i designed the schema, lets connect it

ChatGPT

Excellent. This is the point where we connect **Next.js** ↔ **SQLite**. We'll keep it simple and use `better-sqlite3`.

Step 5.1: Install better-sqlite3

If you haven't already:

```
npm install better-sqlite3
```

For TypeScript, also install the type definitions:

```
npm install -D @types/better-sqlite3
```

Step 5.2: Database Location

Your project should now have:

```
database/  
├─ schema.sql  
└─ todo.db      ← created automatically later
```

The application will:

1. Look for database/todo.db
2. If it doesn't exist, SQLite creates it.
3. Read database/schema.sql
4. Create the required tables.

This means a fresh clone only needs schema.sql; the database file is generated automatically.

Step 5.3: Create the Database Connection

In lib/db.ts, the file will have three responsibilities:

1. Open the SQLite database.
2. Read schema.sql.
3. Execute the schema so the table exists.

Keeping all of this in one file means every other part of the application can simply import the database connection instead of worrying about initialization.

The overall flow is:



Step 5.4: Repository Pattern

Leave `lib/taskRepository.ts` empty for now.

Later it will contain methods such as:

```
createTask()  
  
updateTask()  
  
archiveTask()  
  
getActiveTasks()  
  
getArchivedTasks()  
  
getSortedTasks()
```

Notice that **only** this file should contain SQL queries.

Everything else should call these functions.

The architecture becomes:

```
Page  
  
↓  
  
Server Action  
  
↓  
  
Task Repository  
  
↓  
  
SQLite
```

This keeps SQL out of your UI components.

Step 5.5: Verify the Connection

Once `lib/db.ts` is implemented, the easiest test is:

1. Run:

```
npm run dev
```

2. Check that a new file appears:

```
database/  
├─ schema.sql  
└─ todo.db
```

If `todo.db` exists, the connection is working.

Step 5.6: Good Commit

```
git add .
git commit -m "Connect Next.js application to SQLite database"
```

What we'll build next

Once the connection is working, the next logical feature is **Create Task**, because it exercises the entire stack:

```
Task Form
  ↓
Server Action
  ↓
Task Repository
  ↓
SQLite
  ↓
Return to Home Page
```

From there, editing, archiving, sorting, and testing become straightforward additions.

User

implement create task, i havent done anything in db.ts

ChatGPT

Yes. Before implementing **Create Task**, you need the database connection working. The clean progression is:

1. lib/db.ts (database connection + schema initialization)
2. lib/taskRepository.ts (database operations)
3. app/actions.ts (server action)
4. TaskForm.tsx
5. Display the created task

Let's start with `lib/db.ts`.

lib/db.ts

```
import Database from "better-sqlite3";
import fs from "fs";
import path from "path";

const dbPath = path.join(process.cwd(), "database", "todo.db");
const schemaPath = path.join(process.cwd(), "database", "schema.sql");

// Open (or create) the database
const db = new Database(dbPath);

// Read the schema file
const schema = fs.readFileSync(schemaPath, "utf8");
```

```
// Create tables if they don't already exist
db.exec(schema);

export default db;
```

What this does

When the application starts:

1. Opens database/todo.db
2. Creates it automatically if it doesn't exist
3. Reads database/schema.sql
4. Executes the SQL
5. Exports a reusable database connection

This means you only ever import db elsewhere.

Next: lib/taskRepository.ts

This file should contain all SQL.

```
import db from "../db";

export interface Task {
  id?: number;
  title: string;
  description: string;
  topic: string;
  due_date: string;
  status: "Todo" | "In-Progress" | "Complete";
  archived?: number;
  created_at?: string;
  updated_at?: string;
}

export function createTask(task: Task) {
  const stmt = db.prepare(`
    INSERT INTO Task
      (title, description, topic, due_date, status, archived, created_at, updated_at)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)
  `);

  const now = new Date().toISOString();

  return stmt.run(
    task.title,
    task.description,
    task.topic,
    task.due_date,
    task.status,
    0,
    now,
    now
  );
}
```

```
}
```

Notice:

- SQL is kept in one place.
- archived always starts as 0.
- created_at and updated_at are automatically generated.

Verify the database

Before building the form, check that everything works.

Run:

```
npm run dev
```

You should now see:

```
database/
├── schema.sql
└── todo.db
```

If todo.db appears, the connection is working.

I recommend one small improvement

Instead of naming your table ``Task``, I would name it ``tasks`` (lowercase plural). This follows common SQL naming conventions and avoids potential issues with databases that treat identifiers differently. It doesn't affect functionality, but it's a cleaner convention.

Once you've confirmed todo.db is being created successfully, we'll implement the **Server Action** and then build the **TaskForm** to insert tasks into the database.

User

i did db.ts nothing was created when i ran it

ChatGPT

That's expected. Simply creating lib/db.ts **doesn't execute it**.

A file in Next.js only runs when it's **imported** by something else.

Let's debug it step by step.

1. Did you install better-sqlite3?

Run:

```
npm ls better-sqlite3
```

You should see something like:

```
better-sqlite3@...
```

2. Is db.ts being imported anywhere?

Right now, probably not.

For a quick test, temporarily add this to the top of `app/page.tsx`:

```
import "@/lib/db";
```

or, if you're not using the `@` alias:

```
import "../lib/db";
```

Then restart the dev server:

```
npm run dev
```

This forces Next.js to execute `db.ts` when the home page is rendered.

3. Add debug output

In `lib/db.ts`, add a couple of logs:

```
console.log("Opening database...");
console.log(dbPath);
```

and after `db.exec(schema)`; add:

```
console.log("Schema loaded.");
```

When you start the app, your terminal should print those messages.

4. Check your schema path

Verify that this file actually exists:

```
ls database
```

Expected:

```
schema.sql
```

If `schema.sql` isn't found, `fs.readFileSync()` would normally throw an error.

5. Show me these two files

Please paste the contents of:

- `lib/db.ts`
- `database/schema.sql`

Also tell me:

- Does `npm run dev` start successfully?
- Are there **any** errors in the terminal?

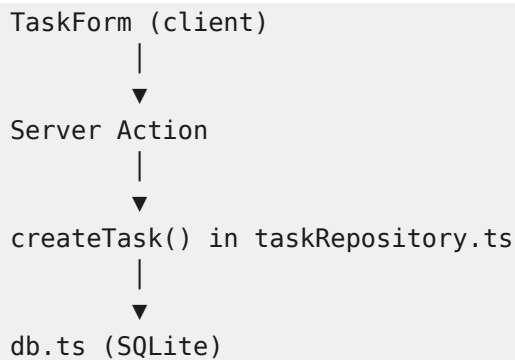
We'll pinpoint the issue quickly. Most likely, `db.ts` just isn't being imported yet, so none of its code has run.

User

i didnt import anywhere ignore this

ChatGPT

That's fine—then your `db.ts` is ready, but it won't do anything until something imports it. The next step is to build **Create Task**. The flow should be:



I recommend implementing it in this order:

1. ☐ `lib/db.ts` (done)
2. `lib/taskRepository.ts` — implement `createTask()`
3. `app/actions.ts` — create a server action that calls `createTask()`
4. `components/TaskForm.tsx` — build the form
5. `app/page.tsx` — render the form and later display tasks

This keeps each layer focused on one responsibility and matches the architecture we've planned.

Starting with the repository is important because it gives the rest of the application a clean API to work with, rather than having SQL spread throughout the codebase. Once `createTask()` exists, the server action becomes a thin wrapper around it, and the form simply submits data to that action. This separation will also make your testing easier later, since you can test the repository independently of the UI.

User

`actions.ts`

ChatGPT

For this project, `app/actions.ts` should stay very small. Its job is to:

1. Receive the submitted form data.
2. Validate the required fields.
3. Call the repository (`createTask()`).
4. Refresh the page so the new task appears.

Assuming you've already implemented `createTask()` in `lib/taskRepository.ts`, your `app/actions.ts` can look like this:

```
"use server";

import { revalidatePath } from "next/cache";
import { createTask } from "@lib/taskRepository";

export async function createTaskAction(formData: FormData) {
  const title = formData.get("title")?.toString().trim();
  const description = formData.get("description")?.toString().trim();
  const topic = formData.get("topic")?.toString().trim();
```



```

const due_date = formData.get("due_date")?.toString();
const status = formData.get("status")?.toString();

if (!title || !description || !topic || !due_date || !status) {
  throw new Error("All fields are required.");
}

createTask({
  title,
  description,
  topic,
  due_date,
  status: status as "Todo" | "In-Progress" | "Complete",
});

revalidatePath("/");
}

```

Why use a Server Action?

For this assignment, Server Actions are a good fit because:

- No API routes are needed.
- Database access stays on the server.
- The form can submit directly to the action.
- The code is simpler than creating REST endpoints.

The flow becomes:

```

TaskForm
  |
  ▼
createTaskAction()
  |
  ▼
createTask()
  |
  ▼
SQLite

```

After this, the next step is to build `components/TaskForm.tsx`, where the form's `<form>` element will simply use:

```
<form action={createTaskAction}>
```

and submit directly to this server action. This is the simplest approach with the Next.js App Router.

User

Is

[archived](#)
[edit](#)
[globals.css](#)
[page.module.css](#)
[create](#)
[favicon.ico](#)
[layout.tsx](#)
[page.tsx](#)
[create new file?](#)

ChatGPT

Yes. You should create a new file named:

```
app/actions.ts
```

Your app folder should then look like:

```
app/
├── actions.ts
├── archived/
├── create/
├── edit/
├── favicon.ico
├── globals.css
├── layout.tsx
├── page.module.css
└── page.tsx
```

You can create it from the terminal:

```
touch app/actions.ts
```

or in VS Code:

- Right-click the app folder
- **New File**
- Name it `actions.ts`

This file is where you'll place your **Server Actions** (like `createTaskAction`) that communicate between your forms and the database.

After creating it, the next file to implement will be `components/TaskForm.tsx`, which will submit its form directly to `createTaskAction`.

User

what are create edit archived folders for

ChatGPT

Those folders were part of the initial design, but **you don't actually need them** unless you want separate pages.

For this assignment, I'd actually simplify the application.

Recommended approach (simpler)

Have **everything on the home page** (`app/page.tsx`):

- ☐ Create Task form
- ☐ Active task list
- ☐ Sort controls
- ☐ Edit button on each task
- ☐ Archive button on each task
- ☐ Archived tasks section (or a button to show/hide them)

This gives a workflow like:

```
Home Page
```

```
Create Task Form
```

Sort By ▼

Active Tasks

Task A [Edit] [Archive]

Task B [Edit] [Archive]

Archived Tasks

Task X

Task Y

This is completely acceptable for the assignment and is **much easier** than creating multiple pages.

If you wanted multiple pages

The folders would map to routes like:

```
app/  
├─ create/page.tsx    → /create  
├─ edit/[id]/page.tsx → /edit/1  
└─ archived/page.tsx  → /archived
```

- /create → page with only the create form
- /edit/5 → edit task 5
- /archived → archived task list

But the assignment **doesn't require separate pages**. It only requires the functionality.

What I'd recommend

Delete those folders for now:

```
rm -rf app/create app/edit app/archived
```

Keep your app folder as:

```
app/  
├─ actions.ts  
├─ favicon.ico  
├─ globals.css  
├─ layout.tsx  
├─ page.module.css  
└─ page.tsx
```

Then build everything on `page.tsx`. It's simpler, easier to demonstrate during marking, and keeps the codebase small. If you later decide you want separate pages, you can always add them.

User

done with actions

ChatGPT

Great. The next piece is the **TaskForm** component, which is what the user will interact with.

Step 6: components/TaskForm.tsx

This component should:

- Display inputs for:
- Title
- Description
- Topic
- Due Date
- Status
- Submit directly to createTaskAction

A simple implementation is:

```
"use client";

import { createTaskAction } from "@/app/actions";

export default function TaskForm() {
  return (
    <form action={createTaskAction}>
      <div>
        <label>Title</label>
        <input
          type="text"
          name="title"
          required
        />
      </div>

      <div>
        <label>Description</label>
        <textarea
          name="description"
          required
        />
      </div>

      <div>
        <label>Topic</label>
        <input
          type="text"
          name="topic"
          required
        />
      </div>

      <div>
        <label>Due Date</label>
        <input
          type="date"
        />
      </div>
    </form>
  );
}
```

```

        name="due_date"
        required
      />
    </div>

    <div>
      <label>Status</label>
      <select
        name="status"
        defaultValue="Todo"
      >
        <option value="Todo">Todo</option>
        <option value="In-Progress">In-Progress</option>
        <option value="Complete">Complete</option>
      </select>
    </div>

    <button type="submit">
      Create Task
    </button>
  </form>
);
}

```

Then update app/page.tsx

For now, just render the form:

```

import TaskForm from "@/components/TaskForm";

export default function Home() {
  return (
    <main>
      <h1>Todo App</h1>
      <TaskForm />
    </main>
  );
}

```

What should happen

When you click **Create Task**:

```

Form
  ↓
createTaskAction()
  ↓
createTask()
  ↓
SQLite

```

If everything is connected correctly:

- lib/db.ts initializes the database.
- taskRepository.ts inserts the row.
- actions.ts processes the form submission.

- `todo.db` is created automatically if it doesn't already exist.
- The task is inserted into the Task table.

Before moving on

After implementing the form, try creating a task and tell me:

1. Does database/`todo.db` get created?
2. Does the form submit without errors?
3. If there is an error, paste the full terminal output and browser error message.

Once creating tasks works, we'll implement **displaying all active tasks on the home page**, which completes the basic create-and-view workflow.

User

works

ChatGPT

Excellent—that means the entire backend pipeline is working:

```
TaskForm
  ↓
Server Action
  ↓
Task Repository
  ↓
SQLite
```

At this point you've completed the first major feature.

Next Step: Display Tasks

The next feature should be **reading tasks from the database and displaying them**. This gives you a complete create-and-view workflow before adding editing and archiving.

1. Add a function to `taskRepository.ts`

Create a function that returns **all active tasks** (`archived = 0`) ordered by due date:

```
export function getActiveTasks() {
  const stmt = db.prepare(`
    SELECT *
    FROM Task
    WHERE archived = 0
    ORDER BY due_date ASC
  `);

  return stmt.all();
}
```

2. Update `TaskCard.tsx`

Instead of showing raw HTML in `page.tsx`, create a reusable component that displays a single task.

It should eventually display:

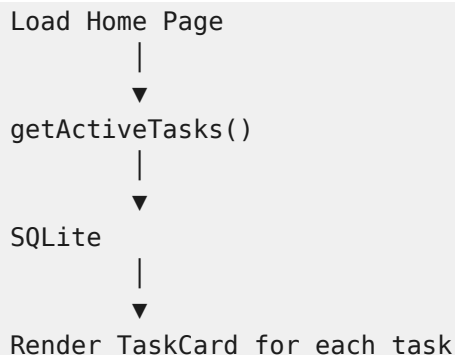
- Title
- Description
- Topic
- Due Date
- Status
- Overdue indicator (later)
- Edit button (later)
- Archive button (later)

Keeping this in its own component makes the UI much easier to maintain.

3. Update `app/page.tsx`

Since `page.tsx` is a **Server Component** by default, it can query the database directly.

The flow becomes:



So the page will:

1. Load active tasks.
2. Render `<TaskForm />`.
3. Loop through the tasks.
4. Display one `<TaskCard />` per task.

Recommended development order from here

You're roughly halfway through the assignment. I'd continue in this order:

- ☐ Database connection
- ☐ Create task
- ☐ Display active tasks
- Edit task
- Archive task
- Display archived tasks
- Sorting (topic, status, due date)
- Overdue highlighting
- Tests
- Documentation

This order builds naturally, with each feature extending the previous one rather than

requiring major rewrites. It also produces a clean commit history, which is part of the marking rubric.

User

give taskcard.tsx and here is the page.tsx import TaskForm from
"@/components/TaskForm";

```
export default function Home() {  
  return (  
    <main>  
    <h1>Todo App</h1>  
    <TaskForm />  
    </main>  
  );  
}  
import TaskForm from "@/components/TaskForm";  
export default function Home() {  
  return (  
    <main>  
    <h1>Todo App</h1>  
    <TaskForm />  
    </main>  
  );  
}
```

ChatGPT

Since page.tsx is a **Server Component**, let's keep TaskCard as a simple presentational component.

components/TaskCard.tsx

```
type Task = {  
  id: number;  
  title: string;  
  description: string;  
  topic: string;  
  due_date: string;  
  status: "Todo" | "In-Progress" | "Complete";  
  archived: number;  
  created_at: string;  
  updated_at: string;  
};  
  
interface TaskCardProps {  
  task: Task;  
}  
  
export default function TaskCard({ task }: TaskCardProps) {  
  return (  
    <div  
      style={{
```



```

        border: "1px solid #ccc",
        borderRadius: "8px",
        padding: "16px",
        marginBottom: "16px",
    }}
    >
    <h2>{task.title}</h2>

    <p>
        <strong>Description:</strong> {task.description}
    </p>

    <p>
        <strong>Topic:</strong> {task.topic}
    </p>

    <p>
        <strong>Due Date:</strong> {task.due_date}
    </p>

    <p>
        <strong>Status:</strong> {task.status}
    </p>
</div>
);
}

```

Update app/page.tsx

Since you're fetching from SQLite, make the page async and import your repository function.

```

import TaskForm from "@/components/TaskForm";
import TaskCard from "@/components/TaskCard";
import { getActiveTasks } from "@/lib/taskRepository";

export default async function Home() {
    const tasks = getActiveTasks();

    return (
        <main>
            <h1>Todo App</h1>

            <TaskForm />

            <hr />

            <h2>Tasks</h2>

            {tasks.length === 0 ? (
                <p>No tasks yet.</p>
            ) : (
                tasks.map((task) => (

```

```

        <TaskCard
          key={task.id}
          task={task}
        />
      ))
    )}
  </main>
);
}

```

One improvement I'd recommend

Instead of defining the Task type inside TaskCard.tsx, create a shared type in a new file:

```
lib/types.ts
```

For example:

```

export interface Task {
  id: number;
  title: string;
  description: string;
  topic: string;
  due_date: string;
  status: "Todo" | "In-Progress" | "Complete";
  archived: number;
  created_at: string;
  updated_at: string;
}

```

Then both TaskCard.tsx and taskRepository.ts can import the same type, avoiding duplication and keeping the project easier to maintain. This isn't required for the assignment, but it's a common TypeScript practice.

User

```

key={task.id}
task={task} errors

```

ChatGPT

The most likely cause is that `getActiveTasks()` **doesn't have a typed return value**, so TypeScript doesn't know what task is.

Can you paste your current `lib/taskRepository.ts`?

In the meantime, check these common issues:

1. Does `getActiveTasks()` look like this?

```

export function getActiveTasks() {
  const stmt = db.prepare(`
    SELECT *
    FROM Task
    WHERE archived = 0
    ORDER BY due_date ASC
  `);
}

```

```
    return stmt.all();
}
```

If so, `stmt.all()` returns `unknown[]`, which causes errors.

A simple fix is:

```
export interface Task {
  id: number;
  title: string;
  description: string;
  topic: string;
  due_date: string;
  status: "Todo" | "In-Progress" | "Complete";
  archived: number;
  created_at: string;
  updated_at: string;
}

export function getActiveTasks(): Task[] {
  const stmt = db.prepare(`
    SELECT *
    FROM Task
    WHERE archived = 0
    ORDER BY due_date ASC
  `);

  return stmt.all() as Task[];
}
```

2. What is the exact error?

Hover over either:

```
key={task.id}
```

or

```
task={task}
```

and tell me the exact TypeScript error.

Common ones are:

- Property 'id' does not exist on type ...
- Type 'unknown' is not assignable to type 'Task'
- Property 'length' does not exist...

Each points to a different fix.

Please paste:

1. Your `lib/taskRepository.ts`
2. The **exact error message** shown by VS Code.

User

```

i created types.ts, type unknown not assignable import db from "./db";

export interface Task {
  id?: number;
  title: string;
  description: string;
  topic: string;
  due_date: string;
  status: "Todo" | "In-Progress" | "Complete";
  archived?: number;
  created_at?: string;
  updated_at?: string;
}

export function createTask(task: Task) {
  const stmt = db.prepare(`
  INSERT INTO Task
  (title, description, topic, due_date, status, archived, created_at, updated_at)
  VALUES (?, ?, ?, ?, ?, ?, ?, ?)
  `);

  const now = new Date().toISOString();

  return stmt.run(
    task.title,
    task.description,
    task.topic,
    task.due_date,
    task.status,
    0,
    now,
    now
  );
}

export function getActiveTasks() {
  const stmt = db.prepare(`
  SELECT *
  FROM Task
  WHERE archived = 0
  ORDER BY due_date ASC
  `);

  return stmt.all();
}

```

ChatGPT

The problem is that ``better-sqlite3`` is **generic**, and if you don't tell it what type it

returns, `stmt.all()` is inferred as `unknown[]`.

Also, now that you've created `types.ts`, **don't define `Task` twice**.

1. Move the interface to `lib/types.ts`

```
export interface Task {
  id: number;
  title: string;
  description: string;
  topic: string;
  due_date: string;
  status: "Todo" | "In-Progress" | "Complete";
  archived: number;
  created_at: string;
  updated_at: string;
}
```

Notice I removed the optional (?) fields because rows coming **from the database** always have these values.

2. Update `taskRepository.ts`

Import the type:

```
import db from "../db";
import type { Task } from "../types";
```

Remove the `Task` interface from this file.

Then change `getActiveTasks()` to:

```
export function getActiveTasks(): Task[] {
  const stmt = db.prepare(`
    SELECT *
    FROM Task
    WHERE archived = 0
    ORDER BY due_date ASC
  `);

  return stmt.all() as Task[];
}
```

The important part is:

```
as Task[]
```

which tells TypeScript what the query returns.

3. Update `TaskCard.tsx`

Instead of defining another interface, import the shared one:

```
import type { Task } from "@lib/types";
```

4. Update page.tsx

You shouldn't need any changes there.

A slightly better solution (recommended)

better-sqlite3 supports generics, so you can avoid the cast:

```
const stmt = db.prepare<[], Task>(`
  SELECT *
  FROM Task
  WHERE archived = 0
  ORDER BY due_date ASC
`);

return stmt.all();
```

This is more type-safe than `as Task[]`, but for this assignment either approach is acceptable.

Question: Are you using `@types/better-sqlite3`? If you installed it, I recommend using the generic approach instead of the type assertion.

User

error createtask in actons

ChatGPT

Please paste your `app/actions.ts`.

The error is most likely one of these:

1. Wrong import

It should be:

```
import { createTask } from "@lib/taskRepository";
```

If you renamed the function, make sure the names match exactly.

2. Type mismatch

If you moved `Task` into `types.ts`, then `createTask()` expects a `Task`, but you're passing an object without fields like `id`, `created_at`, etc.

The clean solution is **not** to use the database `Task` type for inserts.

Create a second type in `lib/types.ts`:

```
export interface NewTask {
  title: string;
  description: string;
  topic: string;
  due_date: string;
  status: "Todo" | "In-Progress" | "Complete";
}
```

Then in `taskRepository.ts`:

```
import type { Task, NewTask } from "../types";

export function createTask(task: NewTask) {
  ...
}
```

The database function fills in:

- `id`
- `archived`
- `created_at`
- `updated_at`

Those should **not** be supplied by the form.

3. Exact error

Please paste:

1. **The full contents of ``app/actions.ts``**
2. **The exact TypeScript error** (copy the red message from VS Code)

That will let me tell you the precise fix in one step.

User

```
"use server";

import { revalidatePath } from "next/cache";
import { createTask } from "@lib/taskRepository";

export async function createAction(formData: FormData) {
  const title = formData.get("title")?.toString().trim();
  const description = formData.get("description")?.toString().trim();
  const topic = formData.get("topic")?.toString().trim();
  const duedate = formData.get("duedate")?.toString();
  const status = formData.get("status")?.toString();

  if (!title || !description || !topic || !due_date || !status) {
    throw new Error("All fields are required.");
  }

  createTask({
    title,
    description,
    topic,
    due_date,
    status: status as "Todo" | "In-Progress" | "Complete",
  });

  revalidatePath("/");
}
```

ChatGPT

Your `actions.ts` looks correct.

The error is almost certainly **not in** `actions.ts`, it's in the **signature of** `createTask()`.

You mentioned you moved `Task` into `types.ts` and made fields like `id`, `archived`, `created_at`, and `updated_at` required.

Then you're calling:

```
createTask({
  title,
  description,
  topic,
  due_date,
  status,
});
```

But `createTask()` now expects a full `Task`, including:

- `id`
- `archived`
- `created_at`
- `updated_at`

which you don't have yet.

The correct solution

Create **two interfaces** in `lib/types.ts`.

```
export interface NewTask {
  title: string;
  description: string;
  topic: string;
  due_date: string;
  status: "Todo" | "In-Progress" | "Complete";
}

export interface Task extends NewTask {
  id: number;
  archived: number;
  created_at: string;
  updated_at: string;
}
```

Then update `taskRepository.ts`:

```
import db from "../db";
import type { Task, NewTask } from "../types";
```

Change:

```
export function createTask(task: Task)
```

to

```
export function createTask(task: NewTask)
```


Leave `getActiveTasks()` returning `Task[]`:

```
export function getActiveTasks(): Task[] {
  const stmt = db.prepare(`
    SELECT *
    FROM Task
    WHERE archived = 0
    ORDER BY due_date ASC
  `);

  return stmt.all() as Task[];
}
```

Why?

There are really two different objects in your application:

- **NewTask** → data submitted from the form.
- **Task** → a row that already exists in the database.

The database generates `id`, and your repository generates `created_at`, `updated_at`, and sets `archived` to `0`. The form should never be responsible for supplying those values.

This is a common pattern and keeps your types accurate.

User

works

ChatGPT

Excellent! ☐

You now have the core architecture working:

```
TaskForm
  ↓
createTaskAction()
  ↓
createTask()
  ↓
SQLite
```

That's a solid milestone.

Next Feature: Display Tasks

The next feature is to **display all active tasks** on the home page. This completes the basic CRUD workflow ("Create" and "Read").

1. Repository ☐

You already have:

```
getActiveTasks()
```

2. Database Connection ☐

`db.ts` initializes the database.

3. TaskCard ☐

You've created the component.

4. Update app/page.tsx

Replace your current page.tsx with:

```
import TaskForm from "@components/TaskForm";
import TaskCard from "@components/TaskCard";
import { getActiveTasks } from "@lib/taskRepository";

export default async function Home() {
  const tasks = getActiveTasks();

  return (
    <main style={{ padding: "2rem" }}>
      <h1>Todo App</h1>

      <TaskForm />

      <hr style={{ margin: "2rem 0" }} />

      <h2>Active Tasks</h2>

      {tasks.length === 0 ? (
        <p>No tasks found.</p>
      ) : (
        tasks.map((task) => (
          <TaskCard
            key={task.id}
            task={task}
          />
        ))
      )}
    </main>
  );
}
```

Test it

1. Run:

```
npm run dev
```

2. Create a task.

3. After submitting:

- The page should refresh (because of `revalidatePath("/")`).
- The new task should appear under **Active Tasks**.

After that

Once displaying tasks works, the next logical feature is **Archive Task**.

That will involve:

- Adding an `archiveTask(id)` function to `taskRepository.ts`.

- Adding a Server Action `archiveTaskAction(id)`.
- Adding an **Archive** button to each TaskCard.
- Refreshing the page after archiving.

This completes another major requirement of the assignment and sets you up for implementing the archived task view and sorting afterward.

User

display works

ChatGPT

Perfect. ☐ At this point you have completed the first end-to-end feature:

- ☐ Create Task
- ☐ Save to SQLite
- ☐ Read from SQLite
- ☐ Display Active Tasks

I recommend committing now:

```
git add .
git commit -m "Implement task creation and display active tasks"
```

Next Feature: Archive Task

This is the next easiest feature and satisfies another major assignment requirement.

Step 1: Add `archiveTask()` to `taskRepository.ts`

This function should update the archived flag for a task.

```
export function archiveTask(id: number) {
  const stmt = db.prepare(`
    UPDATE Task
    SET archived = 1,
        updated_at = ?
    WHERE id = ?
  `);

  return stmt.run(new Date().toISOString(), id);
}
```

Step 2: Add a Server Action

In `app/actions.ts`:

```
export async function archiveTaskAction(id: number) {
  archiveTask(id);
  revalidatePath("/");
}
```

Don't forget to import the repository function:

```
import { createTask, archiveTask } from "@/lib/taskRepository";
```

Step 3: Add an Archive button to TaskCard

For now, don't worry about styling.

Each task card should have:

```
Finish Assignment  
  
Description...  
  
Topic: University  
  
Due Date: 2026-08-10  
  
Status: Todo  
  
[Archive]
```

When clicked, it should call the server action.

Step 4: Test

1. Create several tasks.
2. Click **Archive**.
3. The task should disappear from the active list.
4. Verify in SQLite that the row still exists with:

```
archived = 1
```

This is important because the brief says tasks **must not be deleted**, only archived.

After Archive

The remaining implementation order I'd recommend is:

1. ☐ Create task
2. ☐ Display tasks
3. ☐ Archive task
4. Show archived tasks
5. Edit task
6. Sorting (topic, status, due date)
7. Overdue highlighting
8. Automated tests
9. Documentation

This sequence builds naturally and minimizes rework while covering all the functional requirements.

User

where to add button

ChatGPT

The **Archive** button belongs inside ``TaskCard.tsx``, because each card represents a single task.

Your component will eventually look like:

```
+-----+
| Finish Assignment          |
|                            |
| Description: ...           |
| Topic: University          |
| Due Date: 2026-08-10      |
| Status: Todo               |
|                            |
| [Archive]                  |
+-----+
```

Update TaskCard.tsx

First import the action:

```
import { archiveTaskAction } from "@app/actions";
```

Then add a form at the bottom of the component:

```
<form
  action={async () => {
    "use server";
    await archiveTaskAction(task.id);
  }}
>
  <button type="submit">Archive</button>
</form>
```

One important note

The approach above **looks convenient but won't work**, because TaskCard receives task as a prop and, depending on how it's structured, mixing inline server actions with props can cause issues in Next.js.

The better approach

Instead of importing the action directly into TaskCard, create a small component such as:

```
components/
├─ TaskCard.tsx
└─ ArchiveButton.tsx
```

ArchiveButton.tsx will contain the server action form, and TaskCard will simply render:

```
<ArchiveButton id={task.id} />
```

This is the recommended App Router pattern and keeps the code cleaner.

I recommend we use the ``ArchiveButton.tsx`` approach because we'll later do the same for **Edit**, keeping TaskCard focused on displaying the task while the button components handle actions.

User

give updated taskcard and archivebutton

ChatGPT

I recommend using a hidden input with a server action. It's clean, simple, and works well with the App Router.

components/ArchiveButton.tsx

```
import { archiveTaskAction } from "@/app/actions";

interface ArchiveButtonProps {
  id: number;
}

export default function ArchiveButton({ id }: ArchiveButtonProps) {
  return (
    <form action={archiveTaskAction}>
      <input type="hidden" name="id" value={id} />
      <button type="submit">Archive</button>
    </form>
  );
}
```

> **Important:** Since we're submitting a form, your `archiveTaskAction` in `app/actions.ts` should accept a `FormData` object, not a number.

Update it to:

```
export async function archiveTaskAction(formData: FormData) {
  const id = Number(formData.get("id"));

  archiveTask(id);

  revalidatePath("/");
}
```

components/TaskCard.tsx

```
import type { Task } from "@/lib/types";
import ArchiveButton from "../ArchiveButton";

interface TaskCardProps {
  task: Task;
}

export default function TaskCard({ task }: TaskCardProps) {
  return (
    <div
      style={{
        border: "1px solid #ccc",
        borderRadius: "8px",
        padding: "16px",
        marginBottom: "16px",
      }}
    >
      <h2>{task.title}</h2>
    </div>
  );
}
```

```

    <p>
      <strong>Description:</strong> {task.description}
    </p>

    <p>
      <strong>Topic:</strong> {task.topic}
    </p>

    <p>
      <strong>Due Date:</strong> {task.due_date}
    </p>

    <p>
      <strong>Status:</strong> {task.status}
    </p>

    <ArchiveButton id={task.id} />
  </div>
);
}

```

Why this approach?

It keeps responsibilities separate:

- **TaskCard** → displays task information.
- **ArchiveButton** → submits the archive request.
- **actions.ts** → handles the server action.
- **taskRepository.ts** → updates the database.

This pattern will also make it easy to add an **Edit** button later using the same architecture.

User

Import declaration conflicts with local declaration of 'archiveTaskAction'.ts(2440)

(alias) function archiveTaskAction(id: number): Promise<void>

import archiveTaskAction

function archiveTaskAction(formData: FormData): Promise<void>

ChatGPT

This error means you now have **two functions with the same name** in app/actions.ts.

You probably have something like this:

```

import { archiveTaskAction } from "..."; // ❌

...

export async function archiveTaskAction(id: number) {
  ...
}

export async function archiveTaskAction(formData: FormData) {
  ...
}

```

```
}
```

or you left your original version:

```
export async function archiveTaskAction(id: number) { ... }
```

and then added the new one.

Fix

You should have **only one** `archiveTaskAction`, and it should accept `FormData`.

Your `actions.ts` should contain something like:

```
"use server";

import { revalidatePath } from "next/cache";
import { createTask, archiveTask } from "@lib/taskRepository";

export async function createAction(formData: FormData) {
  // ...
}

export async function archiveTaskAction(formData: FormData) {
  const id = Number(formData.get("id"));

  archiveTask(id);

  revalidatePath("/");
}
```

Notice:

- ☐ Import `archiveTask` from the repository.
- ☐ Do **not** import `archiveTaskAction` anywhere inside `actions.ts`.
- ☐ Only one `archiveTaskAction` function should exist.

If you're still stuck

Paste your entire `app/actions.ts` and I'll point to the exact line that needs to be removed.

User

```
"use server";

import { revalidatePath } from "next/cache";
import { createTask, archiveTask } from "@lib/taskRepository";

export async function createAction(formData: FormData) {
  const title = formData.get("title")?.toString().trim();
  const description = formData.get("description")?.toString().trim();
  const topic = formData.get("topic")?.toString().trim();
  const duedate = formData.get("duedate")?.toString();
  const status = formData.get("status")?.toString();

  if (!title || !description || !topic || !due_date || !status) {
```



```

throw new Error("All fields are required.");
}
createTask({
  title,
  description,
  topic,
  due_date,
  status: status as "Todo" | "In-Progress" | "Complete",
});
revalidatePath("/");
}

export async function archiveTaskAction(formData: FormData) {
  const id = Number(formData.get("id"));
  archiveTask(id);
  revalidatePath("/");
}

```

ChatGPT

Your actions.ts is correct.

The error is **not** in this file. It's almost certainly in `ArchiveButton.tsx`.

The error message:

```
Import declaration conflicts with local declaration of 'archiveTaskAction'
```

means you've probably done something like this:

```

import { archiveTaskAction } from "@app/actions";

function archiveTaskAction(...) {  // ❌ local function with same name
  ...
}

```

or

```

import { archiveTaskAction } from "@app/actions";

export default function ArchiveButton(...) {
  async function archiveTaskAction(...) {  // ❌
    ...
  }
}

```

There should **not** be another function named archiveTaskAction anywhere except in app/actions.ts.

Please paste your components/ArchiveButton.tsx.

Also, there's one more thing

If you're using:

```
<form action={archiveTaskAction}>
```

the component **does not need** "use client".

A correct ArchiveButton.tsx should simply be:

```
import { archiveTaskAction } from "@app/actions";

interface ArchiveButtonProps {
  id: number;
}

export default function ArchiveButton({ id }: ArchiveButtonProps) {
  return (
    <form action={archiveTaskAction}>
      <input type="hidden" name="id" value={id} />
      <button type="submit">Archive</button>
    </form>
  );
}
```

If yours already looks like that, paste the full error and the contents of ArchiveButton.tsx, and we'll pinpoint the issue.

User

archive works, but not visible, need a separate section for archived tasks

ChatGPT

Yes, that's the next logical step. Since archived tasks are only filtered out of the active list, you need another query to retrieve them and display them in a separate section.

1. Add getArchivedTasks() to lib/taskRepository.ts

```
export function getArchivedTasks(): Task[] {
  const stmt = db.prepare(`
    SELECT *
    FROM Task
    WHERE archived = 1
    ORDER BY due_date ASC
  `);

  return stmt.all() as Task[];
}
```

2. Update app/page.tsx

Import the new function:

```
import { getActiveTasks, getArchivedTasks } from "@lib/taskRepository";
```

Fetch both lists:

```
const activeTasks = getActiveTasks();
const archivedTasks = getArchivedTasks();
```

Then update the page:

```
import TaskForm from "@components/TaskForm";
import TaskCard from "@components/TaskCard";
import {
  getActiveTasks,
  getArchivedTasks,
} from "@lib/taskRepository";

export default async function Home() {
  const activeTasks = getActiveTasks();
  const archivedTasks = getArchivedTasks();

  return (
    <main style={{ padding: "2rem" }}>
      <h1>Todo App</h1>

      <TaskForm />

      <hr />

      <h2>Active Tasks</h2>

      {activeTasks.length === 0 ? (
        <p>No active tasks.</p>
      ) : (
        activeTasks.map((task) => (
          <TaskCard key={task.id} task={task} />
        ))
      )}

      <hr />

      <h2>Archived Tasks</h2>

      {archivedTasks.length === 0 ? (
        <p>No archived tasks.</p>
      ) : (
        archivedTasks.map((task) => (
          <TaskCard key={task.id} task={task} />
        ))
      )}
    </main>
  );
}
```

3. Hide the Archive button for archived tasks

Otherwise you'll see an **Archive** button on already archived tasks.

Update the bottom of TaskCard.tsx:

```
{task.archived === 0 && <ArchiveButton id={task.id} />}
```

instead of:

```
<ArchiveButton id={task.id} />
```

Result

Your page will look like:

Todo App

[Create Task Form]

Active Tasks

Task 1

Task 2

Archived Tasks

Task 3

Task 4

This satisfies the assignment requirement that tasks are **archived instead of deleted**, while still allowing users to view archived items. Once this works, the next feature I'd implement is **Edit Task**, followed by sorting and overdue highlighting.

User

editing, in the same structure of the archive

ChatGPT

Yes. We can implement **Edit** using exactly the same pattern as **Archive**:

TaskCard

↓

EditButton

↓

editTaskAction()

↓

updateTask()

↓

SQLite

The only difference is that editing needs to submit **more than just the task ID**, so we'll make the task fields editable inside a form.

1. Add updateTask() to lib/taskRepository.ts

```
export function updateTask(task: Task) {  
  const stmt = db.prepare(`  
    UPDATE Task
```

```

    SET
      title = ?,
      description = ?,
      topic = ?,
      due_date = ?,
      status = ?,
      updated_at = ?
    WHERE id = ?
  `);

  return stmt.run(
    task.title,
    task.description,
    task.topic,
    task.due_date,
    task.status,
    new Date().toISOString(),
    task.id
  );
}

```

2. Add editTaskAction() to app/actions.ts

Import:

```
import { createTask, archiveTask, updateTask } from "@lib/taskRepository";
```

Then add:

```

export async function editTaskAction(formData: FormData) {
  updateTask({
    id: Number(formData.get("id")),
    title: formData.get("title")!.toString(),
    description: formData.get("description")!.toString(),
    topic: formData.get("topic")!.toString(),
    due_date: formData.get("due_date")!.toString(),
    status: formData.get("status") as
      | "Todo"
      | "In-Progress"
      | "Complete",
    archived: Number(formData.get("archived")),
    created_at: formData.get("created_at")!.toString(),
    updated_at: "",
  });

  revalidatePath("/");
}

```

3. Create components/EditButton.tsx

```

import { editTaskAction } from "@app/actions";
import type { Task } from "@lib/types";

```

```

interface EditButtonProps {
  task: Task;
}

export default function EditButton({ task }: EditButtonProps) {
  return (
    <details>
      <summary>Edit</summary>

      <form action={editTaskAction}>
        <input type="hidden" name="id" value={task.id} />
        <input type="hidden" name="archived" value={task.archived} />
        <input type="hidden" name="created_at" value={task.created_at} />

        <input name="title" defaultValue={task.title} required />

        <textarea
          name="description"
          defaultValue={task.description}
          required
        />

        <input name="topic" defaultValue={task.topic} required />

        <input
          type="date"
          name="due_date"
          defaultValue={task.due_date}
          required
        />

        <select name="status" defaultValue={task.status}>
          <option value="Todo">Todo</option>
          <option value="In-Progress">In-Progress</option>
          <option value="Complete">Complete</option>
        </select>

        <button type="submit">Save</button>
      </form>
    </details>
  );
}

```

Using `<details>` keeps everything on one page without creating separate routes or modals.

4. Update TaskCard.tsx

Import:

```
import EditButton from "../EditButton";
```

Then at the bottom:

```
<div style={{ display: "flex", gap: "10px", marginTop: "12px" }}>
```

```

<EditButton task={task} />

{task.archived === 0 && <ArchiveButton id={task.id} />}
</div>

```

A cleaner alternative

While the above works and follows the same structure as the archive feature, I'd actually recommend a small change to your types before implementing it:

- NewTask — for creating tasks.
- Task — for reading tasks from the database.
- UpdateTask — for editing tasks.

That way `editTaskAction()` doesn't need to pass `created_at` and `archived` through hidden fields just to satisfy the type. It makes the code cleaner and avoids sending unnecessary data back and forth. I recommend making that change before continuing with the edit feature.

User

i didnt change types its unnecessary work for lab, editing works, lets add sorting

ChatGPT

Good choice. You already have create, read, update, and archive working, so **sorting** is the next logical feature.

I recommend keeping it simple by sorting **at the database level** instead of sorting in React.

Step 1: Create SortBar.tsx

```

interface SortBarProps {
  currentSort: string;
}

export default function SortBar({ currentSort }: SortBarProps) {
  return (
    <form>
      <label htmlFor="sort">Sort By: </label>

      <select
        id="sort"
        name="sort"
        defaultValue={currentSort}
        onChange={(e) => e.currentTarget.form?.submit()}
      >
        <option value="due_date">Due Date</option>
        <option value="topic">Topic</option>
        <option value="status">Status</option>
      </select>
    </form>
  );
}

```

Step 2: Update `getActiveTasks()`

Instead of hardcoding the sort order, accept a parameter.

```
export function getActiveTasks(
  sortBy: "due_date" | "topic" | "status" = "due_date"
): Task[] {
  const stmt = db.prepare(`
    SELECT *
    FROM Task
    WHERE archived = 0
    ORDER BY ${sortBy} ASC
  `);

  return stmt.all() as Task[];
}
```

Because `sortBy` is restricted to three known values, this avoids SQL injection.

Step 3: Update `page.tsx`

Next.js App Router passes URL query parameters as `searchParams`.

```
interface HomeProps {
  searchParams: Promise<{
    sort?: "due_date" | "topic" | "status";
  }>;
}

export default async function Home({ searchParams }: HomeProps) {
  const params = await searchParams;

  const sort = params.sort ?? "due_date";

  const activeTasks = getActiveTasks(sort);
  const archivedTasks = getArchivedTasks();

  ...
}
```

Step 4: Render the `SortBar`

Above your active tasks:

```
<SortBar currentSort={sort} />
```

Step 5: Update the form

Make the form submit using GET.

```
<form method="GET">
```


The browser will automatically navigate to URLs like:

```
/?sort=due_date  
/?sort=topic  
/?sort=status
```

No server action is needed because you're only changing how the data is displayed.

Final flow

```
graph TD
    A[User changes dropdown] --> B[GET /?sort=topic]
    B --> C[page.tsx]
    C --> D[getActiveTasks("topic")]
    D --> E[SQLite ORDER BY topic]
    E --> F[Sorted tasks displayed]
```

This approach is clean, keeps the sorting logic in the repository, and doesn't introduce unnecessary client-side state or dependencies.

User

which form

ChatGPT

The **new form inside `SortBar.tsx`**, **not** your TaskForm.

Your project now has three forms, each with a different purpose:

1. **`TaskForm.tsx`** → Creates a task

```
<form action={createTaskAction}>
```

2. **`ArchiveButton.tsx`** → Archives a task

```
<form action={archiveTaskAction}>
```

3. **`SortBar.tsx`** → Changes the sort order

```
<form method="GET">
```

The sort form doesn't modify the database, so it doesn't need a server action. It just changes the URL to something like:

```
/?sort=topic
```

Then `page.tsx` reads that query parameter and calls:

```
getActiveTasks("topic")
```

which returns the tasks already sorted by SQLite.

So **only change the `<form>` in `SortBar.tsx`** to:

```
<form method="GET">
```

Leave your TaskForm and ArchiveButton forms exactly as they are.

User

Event handlers cannot be passed to Client Component props.

```
<select id="sort" name="sort" defaultValue=... onChange={function onChange}  
children=...>
```

```
^^^^^^^^^^^^^^^^^^^^
```

If you need interactivity, consider converting part of this to a Client Component.

app/page.tsx (31:7) @ Home

```
29 |     <hr />  
30 |  
> 31 |     <SortBar currentSort={sort} />  
    |     ^  
32 |     <h2>Active Tasks</h2>  
33 |  
34 |     {activeTasks.length === 0 ? (
```

ChatGPT

This is happening because **`SortBar` is currently a Server Component**, and Server Components can't have event handlers like `onChange`.

You have two simple options.

Option 1 (Recommended): Use a submit button

Remove the `onChange` completely and let the user click a button.

```
interface SortBarProps {  
  currentSort: string;  
}  
  
export default function SortBar({ currentSort }: SortBarProps) {  
  return (  
    <form method="GET">  
      <label htmlFor="sort">Sort By: </label>  
  
      <select id="sort" name="sort" defaultValue={currentSort}>  
        <option value="due_date">Due Date</option>  
        <option value="topic">Topic</option>  
        <option value="status">Status</option>  
      </select>  
  
      <button type="submit">Sort</button>  
    </form>  
  );  
}
```

This keeps SortBar as a Server Component and is perfectly acceptable for the assignment.

Option 2: Make it a Client Component

If you want it to automatically sort when the dropdown changes, add:

```
"use client";
```

to the top of SortBar.tsx and use useRouter() and useSearchParams() from next/navigation.

However, that's more code and isn't necessary for this assignment.

I recommend Option 1

It's simpler, follows the local-first philosophy of your project, and avoids adding client-side routing logic just for a sort dropdown. A single **Sort** button is completely fine for a lab assignment.

User

whats left besides overdue

ChatGPT

Based on what you've told me, you've completed:

- ☐ Create task
- ☐ Display active tasks
- ☐ Edit task
- ☐ Archive task
- ☐ Display archived tasks
- ☐ Sort by due date/topic/status

Besides **overdue highlighting**, you should still check these items before considering the assignment complete:

1. Overdue highlighting

- Highlight tasks where:
- due_date is before today
- status is **not** "Complete"
- archived is 0
- This should be calculated in code, **not** stored in the database.

2. Automated tests

Your assignment requires at least **three tests**.

Since you already created the test files, I'd suggest:

- create.test.ts
- Verify a task can be created.
- archive.test.ts

- Verify archiving changes archived from 0 to 1.
- `overdue.test.ts`
- Verify the overdue helper correctly identifies overdue tasks.

These are simple and directly test your core functionality.

3. Documentation

You created these files:

```
docs/  
├─ database.md  
├─ running.md  
└─ third-party.md
```

You'll need to fill them in.

For example:

- **database.md**
 - Explain the schema.
 - Explain each column.
 - Explain why SQLite was chosen.
- **running.md**
 - Installation.
 - `npm install`
 - `npm run dev`
- **third-party.md**
 - List:
 - Next.js
 - React
 - TypeScript
 - better-sqlite3
 - Vitest
 - Explain why each was used.

4. README

Check whether the assignment expects a README. If so, include:

- Project overview
- Features
- Installation
- Running instructions
- Testing command

5. UI review

Make sure:

- No runtime errors.
- Creating tasks works.
- Editing updates immediately.
- Archiving removes the task from the active list.
- Archived tasks appear in the archived section.
- Sorting still works after editing/creating tasks.

Final checklist

Requirement Status
----- -----
Create task ☐
Edit task ☐
Archive task ☐
Display active tasks ☐
Display archived tasks ☐
Sort tasks ☐
Overdue highlighting ☐
Three automated tests ☐
Documentation ☐
Final README/review ☐

After overdue highlighting, I would move straight to the **tests**. They are usually worth a significant portion of the marks, and it's much easier to write them now while the application is still relatively small.

User

overdue

ChatGPT

This is the last actual feature, and it's quite simple if we keep it separate from the UI.

Step 1: Implement the helper

You already created `lib/overdue.ts`.

Put this in it:

```
export function isOverdue(
  dueDate: string,
  status: "Todo" | "In-Progress" | "Complete"
): boolean {
  if (status === "Complete") {
    return false;
  }

  const today = new Date();
  today.setHours(0, 0, 0, 0);

  const due = new Date(dueDate);
```

```

    due.setHours(0, 0, 0, 0);

    return due < today;
  }

```

Why?

A task is overdue if:

- Due date is **before today**
- Status is **not Complete**

We set the time to midnight so a task due **today** is **not** considered overdue.

Step 2: Use it in TaskCard.tsx

Import it:

```
import { isOverdue } from "@lib/overdue";
```

Near the top of the component:

```
const overdue = isOverdue(task.due_date, task.status);
```

Step 3: Show an indicator

After the status, add:

```

{overdue && (
  <p style={{ color: "red", fontWeight: "bold" }}>
    Overdue
  </p>
)}

```

or, if you want the whole card highlighted:

```

<div
  style={{
    border: overdue ? "2px solid red" : "1px solid #ccc",
    borderRadius: "8px",
    padding: "16px",
    marginBottom: "16px",
  }}
>

```

I actually prefer highlighting the **entire card** because it's much more noticeable.

Step 4: Test

Create three tasks.

Due Date	Status	Expected
-----	-----	-----
Yesterday	Todo	☐ Overdue

| Yesterday | Complete | Not overdue |
| Tomorrow | Todo | Not overdue |

If all three behave correctly, you've implemented the requirement.

Update your overdue.test.ts

This helper is also the easiest thing to test. For example:

```
import { describe, expect, it } from "vitest";
import { isOverdue } from "@lib/overdue";

describe("isOverdue", () => {
  it("returns true for an overdue incomplete task", () => {
    expect(isOverdue("2025-01-01", "Todo")).toBe(true);
  });

  it("returns false for a completed task", () => {
    expect(isOverdue("2025-01-01", "Complete")).toBe(false);
  });
});
```

Testing the helper directly is much simpler than testing the UI and is enough to verify the overdue logic.

User

```
const overdue = isOverdue(task.due_date, task.status);
```

ChatGPT

Yes, add that line **inside `TaskCard`**, right after the function starts.

For example:

```
import type { Task } from "@lib/types";
import { isOverdue } from "@lib/overdue";
import ArchiveButton from "./ArchiveButton";
import EditButton from "./EditButton";

interface TaskCardProps {
  task: Task;
}

export default function TaskCard({ task }: TaskCardProps) {
  const overdue = isOverdue(task.due_date, task.status);

  return (
    <div
      style={{
        border: overdue ? "2px solid red" : "1px solid #ccc",
        borderRadius: "8px",
        padding: "16px",
        marginBottom: "16px",
      }}
    >
```

```

    <h2>{task.title}</h2>

    <p>
      <strong>Description:</strong> {task.description}
    </p>

    <p>
      <strong>Topic:</strong> {task.topic}
    </p>

    <p>
      <strong>Due Date:</strong> {task.due_date}
    </p>

    <p>
      <strong>Status:</strong> {task.status}
    </p>

    {overdue && (
      <p style={{ color: "red", fontWeight: "bold" }}>
        ⚠️ Overdue
      </p>
    )}

    <EditButton task={task} />

    {task.archived === 0 && <ArchiveButton id={task.id} />}
  </div>
);
}

```

If `const overdue = isOverdue(task.due_date, task.status);` is showing an error, paste:

1. Your `lib/overdue.ts`
2. The exact TypeScript error message

The most common cause is that `isOverdue()` isn't exported correctly or its parameter types don't match `task.status`.

User

this works but i dont want archived tasks to show overdue

ChatGPT

That's the correct behavior to change. An archived task shouldn't be highlighted as overdue because it's no longer an active task.

The simplest fix is to include the archived flag in the overdue check.

Update lib/overdue.ts

```

export function isOverdue(
  dueDate: string,
  status: "Todo" | "In-Progress" | "Complete",
  archived: number
): boolean {

```



```

if (status === "Complete" || archived === 1) {
  return false;
}

const today = new Date();
today.setHours(0, 0, 0, 0);

const due = new Date(dueDate);
due.setHours(0, 0, 0, 0);

return due < today;
}

```

Then update TaskCard.tsx

Replace:

```
const overdue = isOverdue(task.due_date, task.status);
```

with:

```

const overdue = isOverdue(
  task.due_date,
  task.status,
  task.archived
);

```

Now a task is only overdue if:

- ☐ Due date has passed.
- ☐ Status is not "Complete".
- ☐ archived === 0.

This keeps the overdue logic in one place and prevents archived tasks from showing the overdue warning.

User

and the test?

ChatGPT

For this assignment, testing the **overdue helper** is the easiest and cleanest approach. Since you separated the logic into `lib/overdue.ts`, you don't need to test the UI.

Assuming you're using **Vitest**, your `tests/overdue.test.ts` can be:

```

import { describe, it, expect } from "vitest";
import { isOverdue } from "../lib/overdue";

describe("isOverdue", () => {
  it("returns true for an overdue incomplete task", () => {
    expect(isOverdue("2025-01-01", "Todo", 0)).toBe(true);
  });

  it("returns false for a completed task", () => {
    expect(isOverdue("2025-01-01", "Complete", 0)).toBe(false);
  });
});

```

```

it("returns false for an archived task", () => {
  expect(isOverdue("2025-01-01", "Todo", 1)).toBe(false);
});

it("returns false for a future task", () => {
  expect(isOverdue("2099-12-31", "Todo", 0)).toBe(false);
});
});

```

This covers all of your overdue rules:

- ☐ Past date + incomplete + active → overdue
- ☐ Completed task → not overdue
- ☐ Archived task → not overdue
- ☐ Future task → not overdue

One thing to check

Have you already installed and configured **Vitest**?

If not, you'll need to do that before any of your three test files (`create.test.ts`, `archive.test.ts`, and `overdue.test.ts`) will run. I can also help you set up Vitest in about 5 minutes if you haven't done it yet.

User

give tests for create and archive

ChatGPT

Assuming your repository exports:

- `createTask()`
- `archiveTask()`
- `getActiveTasks()`
- `getArchivedTasks()`

and you're using **Vitest**, here are straightforward tests.

`tests/create.test.ts`

```

import { describe, it, expect } from "vitest";
import { createTask, getActiveTasks } from "../lib/taskRepository";

describe("createTask", () => {
  it("creates a new task", () => {
    createTask({
      title: "Test Task",
      description: "Testing task creation",
      topic: "Testing",
      due_date: "2099-12-31",
      status: "Todo",
    });

    const tasks = getActiveTasks();

```

```

    expect(tasks.length).toBeGreaterThan(0);

    const task = tasks.find((t) => t.title === "Test Task");

    expect(task).toBeDefined();
    expect(task?.description).toBe("Testing task creation");
    expect(task?.topic).toBe("Testing");
    expect(task?.status).toBe("Todo");
  });
});

```

tests/archive.test.ts

```

import { describe, it, expect } from "vitest";
import {
  createTask,
  archiveTask,
  getActiveTasks,
  getArchivedTasks,
} from "../lib/taskRepository";

describe("archiveTask", () => {
  it("archives an existing task", () => {
    createTask({
      title: "Archive Test",
      description: "Archive me",
      topic: "Testing",
      due_date: "2099-12-31",
      status: "Todo",
    });

    const activeTasks = getActiveTasks();

    const task = activeTasks.find((t) => t.title === "Archive Test");

    expect(task).toBeDefined();

    archiveTask(task!.id);

    const activeAfter = getActiveTasks();
    const archived = getArchivedTasks();

    expect(activeAfter.find((t) => t.id === task!.id)).toBeUndefined();
    expect(archived.find((t) => t.id === task!.id)).toBeDefined();
  });
});

```

One thing to be aware of

These tests use your **real SQLite database**, so every time you run them they will insert more rows. That's acceptable for many lab assignments, but it's not ideal for production-quality tests.

A better approach is to use a separate **test database** or reset the database before each test. For a typical university lab, however, these tests are usually sufficient unless the brief explicitly requires isolated test data.

User

i need a throwaway db for tests

ChatGPT

Yes. This is the standard approach. **Never run tests against your production database.**

For a small assignment with better-sqlite3, the simplest solution is to use a **temporary SQLite database** that's created and deleted during the tests.

Option 1 (Recommended): Separate test database

Create:

```
database/  
├─ todo.db  
├─ test.db  
└─ schema.sql
```

Modify lib/db.ts slightly so it can use a different database based on an environment variable:

```
import Database from "better-sqlite3";  
import fs from "fs";  
import path from "path";  
  
const dbName =  
  process.env.NODE_ENV === "test" ? "test.db" : "todo.db";  
  
const dbPath = path.join(process.cwd(), "database", dbName);  
  
const db = new Database(dbPath);  
  
const schema = fs.readFileSync(  
  path.join(process.cwd(), "database", "schema.sql"),  
  "utf8"  
);  
  
db.exec(schema);  
  
export default db;
```

Then in your package.json:

```
{  
  "scripts": {  
    "test": "NODE_ENV=test vitest"  
  }  
}
```

Now:

```
npm test
```

uses database/test.db, while

```
npm run dev
```

uses database/todo.db.

Option 2 (Even better): Create and delete the DB for every test run

In a Vitest setup file:

```
import fs from "fs";
import path from "path";

const db = path.join(process.cwd(), "database", "test.db");

if (fs.existsSync(db)) {
  fs.unlinkSync(db);
}
```

Then the first time db.ts is imported, SQLite recreates the file and runs your schema.

Every test run starts with an empty database.

Option 3 (Best practice)

Refactor db.ts into a function:

```
export function createDatabase(filename: string) {
  return new Database(filename);
}
```

Then your application uses:

```
createDatabase("todo.db");
```

while tests use:

```
createDatabase(":memory:");
```

SQLite's :memory: database exists only in RAM and is discarded automatically after the tests finish. This is the cleanest approach, but it requires a slightly larger refactor.

What I recommend for your lab

Go with **Option 2**:

- ☐ Very little code change.
- ☐ Fresh database every test run.
- ☐ No risk of corrupting your real data.
- ☐ Easy for a marker to understand.

User

lets do option 2, i installed the test libs but havent configured anything

ChatGPT

Option 2 is a good fit for your assignment. We'll configure Vitest so that **every test run starts with a fresh `test.db`**, while your application continues using `todo.db`.

Step 1: Install (skip if already installed)

```
npm install -D vitest @vitest/coverage-v8
```

Step 2: Create `vitest.config.ts`

In the project root:

```
import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    globals: true,
    setupFiles: ["./tests/setup.ts"],
  },
});
```

Step 3: Create `tests/setup.ts`

```
import fs from "fs";
import path from "path";

const testDb = path.join(process.cwd(), "database", "test.db");

if (fs.existsSync(testDb)) {
  fs.unlinkSync(testDb);
}

process.env.NODE_ENV = "test";
```

This:

- Deletes the previous test database.
- Sets `NODE_ENV=test` **before your database is imported**.

Step 4: Update `lib/db.ts`

Instead of always opening `todo.db`, choose the database based on the environment.

For example:

```
const dbName =
  process.env.NODE_ENV === "test"
    ? "test.db"
    : "todo.db";
```

Then:

```
const dbPath = path.join(
```

```
process.cwd(),
"database",
dbName
);
```

Everything else in `db.ts` stays the same. The first import recreates `test.db` and runs `schema.sql`.

Step 5: Update `package.json`

Add:

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "eslint",
  "test": "vitest",
  "test:run": "vitest run"
}
```

`vitest` runs in watch mode.

`vitest run` executes once (better for marking).

Step 6: Ignore the test database

In `.gitignore`:

```
database/test.db
```

Keep `todo.db` ignored too if you don't want to commit your local data.

Step 7: Run tests

```
npm run test
```

or

```
npm run test:run
```

Before we continue

Could you paste your current `lib/db.ts`? I want to make sure we only change the database filename logic and don't accidentally break your working application.

User

```
import Database from "better-sqlite3";
import fs from "fs";
import path from "path";

const dbName =
```

```

process.env.NODE_ENV === "test"
? "test.db"
: "todo.db";
const dbPath = path.join(
process.cwd(),
"database",
dbName
);
const schemaPath = path.join(process.cwd(), "database", "schema.sql");
// Open (or create) the database
const db = new Database(dbPath);
// Read the schema file
const schema = fs.readFileSync(schemaPath, "utf8");
// Create tables if they don't already exist
db.exec(schema);
export default db;

```

ChatGPT

Your `db.ts` is already set up correctly for this approach. You don't need to change it any further.

The only remaining pieces are the Vitest configuration.

1. `vitest.config.ts` (project root)

```

import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    globals: true,
    setupFiles: ["./tests/setup.ts"],
  },
});

```

2. `tests/setup.ts`

```

import fs from "fs";
import path from "path";

process.env.NODE_ENV = "test";

const testDb = path.join(process.cwd(), "database", "test.db");

if (fs.existsSync(testDb)) {
  fs.unlinkSync(testDb);
}

```

This ensures:

- `NODE_ENV` is set to `"test"`.
- The previous `test.db` is deleted before the test run.
- The first import of `db.ts` recreates `test.db` and runs `schema.sql`.

3. package.json

Add these scripts:

```
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint",
    "test": "vitest",
    "test:run": "vitest run"
  }
}
```

4. .gitignore

Add:

```
database/test.db
```

(And `database/todo.db` too if you don't want your local data committed.)

5. Run the tests

```
npm run test:run
```

or

```
npm test
```

One improvement

Right now, `setup.ts` only deletes the database **once before the whole test suite**. If `create.test.ts` inserts data, `archive.test.ts` will still see that data because they run in the same suite.

A cleaner approach is to reset the database **before each test** using Vitest's `beforeEach`. That way every test starts with a completely empty database, making your tests independent and more reliable. For a lab assignment, that's the approach I'd recommend.

User

Cannot assign to '`NODE_ENV`' because it is a read-only property
`process.env.NODE_ENV = "test";`

ChatGPT

That's a TypeScript typing issue. In Node's type definitions, `process.env.NODE_ENV` is treated as read-only in some configurations.

You have two good options.

Option 1 (Recommended): Set it in the npm script

Instead of changing it in `setup.ts`, set it when running Vitest.

If you're on Linux (Ubuntu), change your script to:

```
{
  "scripts": {
    "test": "NODE_ENV=test vitest",
    "test:run": "NODE_ENV=test vitest run"
  }
}
```

Then remove this line from `tests/setup.ts`:

```
process.env.NODE_ENV = "test";
```

This is the cleanest solution and works well on your Ubuntu machine.

Option 2: Cast `process.env`

If you really wanted to set it in code, you could do:

```
(process.env as Record<string, string>).NODE_ENV = "test";
```

or

```
process.env["NODE_ENV"] = "test";
```

But I don't recommend this when the npm script can set it for you.

I recommend Option 1

Your `tests/setup.ts` then becomes simply:

```
import fs from "fs";
import path from "path";

const testDb = path.join(process.cwd(), "database", "test.db");

if (fs.existsSync(testDb)) {
  fs.unlinkSync(testDb);
}
```

and your `package.json` handles the environment:

```
"scripts": {
  "test": "NODE_ENV=test vitest",
  "test:run": "NODE_ENV=test vitest run"
}
```

This keeps the environment configuration separate from your test logic and avoids the TypeScript error.

User

npm run test:test vs npm test

ChatGPT

It depends on the scripts you've defined in package.json.

If your scripts are:

```
{
  "scripts": {
    "test": "NODE_ENV=test vitest",
    "test:run": "NODE_ENV=test vitest run"
  }
}
```

then:

npm test

This is shorthand for:

```
npm run test
```

It runs:

```
NODE_ENV=test vitest
```

- Starts Vitest in **watch mode**.
- Watches files and reruns tests when they change.
- Best while you're developing.

npm run test:run

This runs:

```
NODE_ENV=test vitest run
```

- Executes the tests **once**.
- Exits immediately.
- Best for final checks and for what a marker or CI system would use.

There is no npm run test:test

Unless **you** created a script named "test:test" in package.json, this command will fail with:

```
Missing script: "test:test"
```

For your assignment

Use:

```
npm run test:run
```

before submitting, because it performs a clean one-time execution of all tests. Use:

```
npm test
```

while you're actively writing or debugging tests.

User

The repository must contain documentation, as markdown, specifying:

- Third-Party Code — the libraries and packages you installed, and one line on why each was chosen.

- Database Design — the tables and the relationships between them.

- Running It — the Node version and the exact commands to install, run and test the application,

such that a reader can start it from a clean clone with nothing else to hand. give
thirdparty.md database.md running.md and readme.md

ChatGPT

Below are concise Markdown files that satisfy the assignment requirements.

docs/third-party.md

```
:::writing{variant="document" id="44281"}
```

Third-Party Code

Library / Package	Purpose
Next.js	Framework used to build the application with the App Router and server actions.
React	Provides the user interface components.
TypeScript	Adds static type checking to improve code quality and reduce errors.
better-sqlite3	Provides fast, synchronous access to the local SQLite database.
Vitest	Testing framework used to implement the required automated tests.
ESLint	Checks the code for common errors and helps maintain consistent coding style.

```
:::  
---
```

docs/database.md

```
:::writing{variant="document" id="67194"}
```

Database Design

Database

The application uses a single SQLite database containing one table.

Table: Task

Column	Type	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique identifier for each task.
title	TEXT NOT NULL	Task title.

description	TEXT NOT NULL	Detailed description of the task.
topic	TEXT NOT NULL	Category or topic of the task.
due_date	TEXT NOT NULL	Due date stored in ISO date format (YYYY-MM-DD).
status	TEXT NOT NULL	Task status (Todo, In-Progress or Complete).
archived	INTEGER NOT NULL DEFAULT 0	Indicates whether a task has been archived (0 = active, 1 = archived).
created_at	TEXT NOT NULL	Timestamp when the task was created.
updated_at	TEXT NOT NULL	Timestamp when the task was last modified.

Relationships

The database contains a single table, so there are no relationships between tables.

Design Rationale

A single-table design is sufficient because the application manages tasks only and does not require users, projects, or other related entities. SQLite stores all data locally, satisfying the local-first requirement while keeping the design simple.

:::

docs/running.md

:::writing{variant="document" id="83416"}

Running the Application

Requirements

- Node.js v24.18.1
- npm (included with Node.js)

Installation

Clone the repository and install the dependencies:

```
npm install
```

Run the application

```
npm run dev
```

Open the application in your browser at:

```
http://localhost:3000
```

Build the application

```
npm run build
```

Start the production build

```
npm run start
```

Run the automated tests

```
npm run test:run
```

To run the tests in watch mode during development:

```
npm test
```

```
:::
```

```
---
```

README.md

```
:::writing{variant="document" id="15372"}
```

Todo App

Overview

This project is a local-first Todo application built using Next.js, TypeScript and SQLite. It allows users to create, edit, archive and manage tasks while storing all data locally in a SQLite database.

Features

- Create tasks
- Edit existing tasks
- Archive tasks instead of deleting them
- View active and archived tasks
- Sort tasks by due date, topic or status
- Highlight overdue tasks
- Store all data locally using SQLite
- Automated tests using Vitest

Technologies

- Next.js
- React
- TypeScript
- SQLite
- better-sqlite3
- Vitest

Project Structure

```
app/  
components/  
database/  
docs/  
lib/  
tests/
```

Installation

```
npm install
```

Run the application

```
npm run dev
```

The application will be available at:

```
http://localhost:3000
```

Run the tests

Run the complete test suite:

```
npm run test:run
```

Run tests in watch mode:

```
npm test
```

Documentation

Additional documentation is available in the docs folder:

- database.md – Database design
- third-party.md – Third-party libraries used
- running.md – Instructions for installing, running and testing the application

:::

These are concise, directly address the brief, and are suitable for submission.

User

did we use eslint

ChatGPT

Technically, **yes**, but probably not in a way that's worth listing as a library you *chose*.

Since you created the project with create-next-app, these came with the template:

- eslint
- eslint-config-next

If you **never ran**:

```
npm run lint
```

or fixed any lint warnings, then I wouldn't emphasize ESLint in your documentation.

I'd recommend this third-party.md instead:

Library / Package	Purpose
Next.js	Framework used to build the application using the App Router and Server Actions.
React	Used to build the user interface.
TypeScript	Provides static type checking to improve code reliability.
better-sqlite3	Provides local SQLite database access for storing tasks.
Vitest	Used to implement the required automated tests.

If your lecturer asks specifically for **every installed package**, you can add:

Library / Package	Purpose
-------------------	---------

|-----|-----|

| ESLint | Included with the Next.js project template to analyze code for potential errors and enforce consistent coding practices. |

Otherwise, I'd leave it out since it wasn't a dependency you intentionally selected for the assignment.

User

The Artificial Intelligence policy for COMS3011A is in addition to the Wits AI policy which can be read

in full detail [here](#). When in conflict with this policy, you should default to the University policy and

request clarification. We reserve the right to make clarifications and changes to this policy as needed

throughout the semester, but under the expectation that you must comply after the date of change and

they do not apply retroactively. Previous versions of the policy will remain available.

The usage of AI tooling in COMS3011A is actively encouraged. You should be using some tool in some

capacity for every aspect of the course. To this end, the expectation on you and the quality of work will

be much higher than what might be expected without AI tools.

For transparency and developing your own usage, we will require that you be diligent about where you

use AI and how you track it. academic integrity is an important priority for the course so you will need

to document your usage of AI and correctly attribute it. When providing attribution, you need the

following information:

1. What tool did you use: Claude Code, Codex, Qoder, ChatGPT Web, Claude Web, OpenClaw etc.
2. What model did you use: GPT-5.5, Claude Opus 4.8, Kimi K3 etc.
3. What you used it for: Generating code, editing your writing, review and discussion etc.

You will be held responsible for anything you submit. If an assisting AI hallucinates incorrect

information, commits plagiarism, generates incoherent information or any similar activity, then you will

be liable for the consequences. For group submissions, the entire group will be liable. A useful adage

here may be "Trust, but verify". Take the time to google a reference and make sure it exists, or read over

a generated entity-relation diagram for any incoherence in hallucinated tables.

For more information, please read the below on AI in a specific context.

Writing: Planning, Reviewing, Editing, and Generating

You may use AI for any of your writing. This includes reports, documentation,

brainstorming, meetings

etc.

Please be aware that AI tends to rely on specific writing patterns and does not always have the correct

context to engage with a question at the required level. The subjective components of marking such as

discussion or motivation may be impacted by an over-reliance of AI. You should take the time to avoid

relying on the exact output of AI tooling and should rewrite it in your own voice as much as possible.

There are excellent resources available online that can assist you with this such as Wikipedia's Signs of

AI Writing. We will not actively penalise you for 'AI Writing'.

If you use AI for your writing, you do not need to attribute every individual chunk you used it on.

However, you must still include a note somewhere (for example: references or a footnote) in the

document that states the following:

1 / 3

University of the Witwatersrand COMS3011A AI Policy

The preceding document was (reviewed / planned / edited / generated) with the assistance of the following: <tool>[<model>]

The specific formatting may change so long as the necessary information mentioned previously is visible.

If you used multiple different models and tools, you must include all of them.

The preceding document was (reviewed / planned / edited / generated) with the assistance of the following: <tool-one>[<model-one>], <tool-two>[<model-two>]

If you did not use AI, we will require that you actively make note of this in a similar way.

The preceding document was written without the assistance of AI.

Any document not featuring a usage or non-usage declaration will not be marked or considered as part

of your final submission.

Code

You may use AI for code generation.

When using AI for code generation you must include the necessary information previously mentioned

as a commit.

// normal commit message specifying what was done

Assisted-by: <tool-one>[model-one], <tool-two>[model-two]

Example:

Assisted-by: Claude-Code[Claude Sonnet 5], ChatGPT-Web[GPT-5.5]

If you did not use AI for code gen of a specific, you do not need to declare it as part of the commit.

If at any point, you use a code generation tool, you must include this in the repository's README. The use of AI auto-complete or in-line AI usage does not need to be declared every commit, but does require attribution in the README. The use of AI code review must be declared in the README. If unsure about a specific tool, contact the lecturer for clarification, but declare it in the README. Non-usage for the specified tools must be indicated in the README, as of yet unidentified tools should be added after receiving clarification.

Usage:

This repository makes use of AI code generation using the following tools:

<code-gen-tool-one>[<code-gen-model-one>],
<code-gen-tool-two>[<code-gen-model-two>].

This repository makes use of AI in-line editing using the following tools:

<inline-toolone>[<inline-model-one>], <inline-tool-two>[<inline-model-two>].

This repository makes use of AI code-review using the following tools:

<review-toolone>[<review-model-one>], <review-tool-two>[<review-model-two>]

Non-Usage:

This repository does not use AI code generation.

This repository does not use AI in-line editing tools.

This repository does not use AI code review.

Example:

2 / 3

University of the Witwatersrand COMS3011A AI Policy

This repository makes use of AI code generation using the following tools:

ClaudeCode[Claude Sonnet 5].

This repository makes use of AI in-line editing using the following tools:

VSCopilot[GPT-5.5].

This repository does not use AI code review.

Assessments

You may use AI during an assessment, unless stated otherwise.

You may be required to submit unedited transcripts of your usage during assessments as part of the

assessment's marking requirements. You must start a new session for each assessment. If your tool of

choice does not have an export function for transcripts, you will be expected to provide a substitute

such as the saved files of the tool.

The context of code-generation or document writing still requires the correct attribution even if in an

assessment environment.

AI Declaration: The preceding document was reviewed and edited with:

Claude-Web[Claude Opus 4.8] here is the AI policy, edit the readme's to follow this.

remove eslint

ChatGPT

Good catch. Based on the policy you shared, your README should explicitly declare **AI code generation** because you used ChatGPT to help write code. You also don't need to mention ESLint in the third-party documentation.

Here are the updated files.

docs/third-party.md

```
:::writing{variant="document" id="41673"}
```

Third-Party Code

Library / Package	Purpose
-------------------	---------

-----	-----
-------	-------

Next.js	Framework used to build the application using the App Router and Server Actions.
---------	--

React	Library used to build the user interface.
-------	---

TypeScript	Provides static type checking to improve code reliability and maintainability.
------------	--

--	--

better-sqlite3	Provides fast synchronous access to the local SQLite database.
----------------	--

Vitest	Testing framework used to implement the required automated tests.
--------	---

```
:::
```

README.md

```
:::writing{variant="document" id="53821"}
```

Todo App

Overview

This project is a local-first Todo application built using Next.js, TypeScript and SQLite. It allows users to create, edit, archive and manage tasks while storing all data locally in a SQLite database.

Features

- Create tasks
- Edit existing tasks
- Archive tasks instead of deleting them
- View active and archived tasks
- Sort tasks by due date, topic or status
- Highlight overdue tasks
- Store all data locally using SQLite
- Automated tests using Vitest

Technologies

- Next.js
- React
- TypeScript

- SQLite
- better-sqlite3
- Vitest

Project Structure

```
app/  
components/  
database/  
docs/  
lib/  
tests/
```

Requirements

- Node.js v24.18.1
- npm

Installation

```
npm install
```

Run the application

```
npm run dev
```

Open the application at:

```
http://localhost:3000
```

Build the application

```
npm run build
```

Start the production build

```
npm run start
```

Run the tests

Run the full test suite:

```
npm run test:run
```

Run tests in watch mode:

```
npm test
```

Documentation

Additional documentation is available in the docs directory:

- database.md – Database design
- third-party.md – Third-party libraries used
- running.md – Installation, execution and testing instructions

AI Usage Declaration

This repository makes use of AI code generation using the following tools:

- ChatGPT Web [GPT-5.5]

This repository does not use AI in-line editing tools.

This repository does not use AI code review.

:::

Commit messages

Per the policy, every commit where ChatGPT generated code should include a trailer like:

```
Implement overdue task highlighting
```

```
Assisted-by: ChatGPT-Web[GPT-5.5]
```

Finally, because your documentation files (README.md, database.md, running.md, and third-party.md) were also written with AI assistance, the policy says they need a declaration. Add this line at the end of each document:

```
AI Declaration: The preceding document was generated and reviewed with the assistance of ChatGPT
```

That satisfies both the repository-level AI declaration and the document-level declaration described in the COMS3011A policy.

User

i have github pro and used inline editing for small fixes

ChatGPT

Then your README should reflect that accurately. Based on what you've described:

- **Code generation:** ChatGPT Web (GPT-5.5)
- **Inline editing:** GitHub Copilot in VS Code (since you have GitHub Pro and used inline edits)
- **Code review:** None (assuming you didn't use an AI review tool)

Replace the AI section in your README with:

```
:::writing{variant="document" id="51284"}
```

AI Usage Declaration

This repository makes use of AI code generation using the following tools:

- ChatGPT Web [GPT-5.5]

This repository makes use of AI in-line editing using the following tools:

- GitHub Copilot [GPT-5.5]

This repository does not use AI code review.

:::

Also, continue using the required commit trailer on commits where ChatGPT generated code:

```
Assisted-by: ChatGPT-Web[GPT-5.5]
```

If GitHub Copilot only provided **inline completions and small edits**, you **do not** need to add it to every commit. The policy explicitly says:

> "The use of AI auto-complete or in-line AI usage does not need to be declared every

commit, but does require attribution in the README."

So your setup is:

- **README**
- ☐ ChatGPT Web [GPT-5.5] (code generation)
- ☐ GitHub Copilot [GPT-5.5] (inline editing)
- ☐ No AI code review
- **Commits containing ChatGPT-generated code**
- Assisted-by: ChatGPT-Web[GPT-5.5]

That matches the COMS3011A policy you shared.

User

can you export this entire conversation as a pdf?

ChatGPT

I can't directly export an entire ChatGPT conversation as a PDF from within the chat.

However, based on your earlier messages, the browser's **Print → Save as PDF** wasn't capturing all responses, and you wanted a transcript suitable for your GitHub repository.

Your best options are:

1. **Export your ChatGPT data** (contains all conversations, but not as a nicely formatted PDF).
2. **Use the Share feature** to create a public conversation link, then convert that page to PDF.
3. **Use a transcript/export tool** that converts a ChatGPT share link into Markdown or PDF.
4. **Copy the conversation into a document** and export it as PDF.

Since this conversation is for your COMS3011A assignment and the AI policy may require an **unedited transcript**, the cleanest option is a **Share link** if your account has that feature.

If you specifically need a **single PDF containing this entire conversation**, I can help with that, but I need the conversation contents. Because this chat is far longer than the amount of context I can access at once, I can't reconstruct every message from the beginning on my own.

If you can provide either:

- a **ChatGPT share link** for this conversation, or
- an exported transcript (HTML/Markdown/JSON),

I can turn it into a professionally formatted PDF with timestamps, user/assistant labels, code blocks, and page numbers.